



Select → 무엇을 보는가

From → 어디서 보는가

where / group by / having / limit → 어떻게

SQL 기초 문법



SQL

기본 문법 익히기



04-1 SELECT 문으로 데이터 조회하기

04-2 WHERE 문으로 조건에 맞는 데이터 조회하기

04-3 ORDER BY 문으로 데이터 정렬하기

04-4 와일드카드로 문자열 조회하기

04-5 GROUP BY 문으로 데이터 묶기

04-6 테이블 생성 및 조작하기

04-7 MySQL의 데이터 유형 정리하기

- SELECT 문은 데이터베이스에서 데이터를 조회하는 구문임
- SELECT 문은 자주 사용하지만 시스템 성능에 많은 영향을 미치므로 주의해서 사용해야 함
- MySQL 공식 문서에서 보는 이는 SELECT 문의 주요 바뀔

Do it! MySQL 공식 문서에 소개된 SELECT 문

```

SELECT
    [ALL | DISTINCT | DISTINCTROW]
    [HIGH_PRIORITY]
    [STRAIGHT_JOIN]
    [SQL_SMALL_RESULT] [SQL_BIG_RESULT] [SQL_BUFFER_RESULT]
    [SQL_NO_CACHE] [SQL_CALC_FOUND_ROWS]
    select_expr [, select_expr] ...
    [into_option]
    [FROM table_references
        [PARTITION partition_list]]
    [WHERE where_condition]
    [GROUP BY {col_name | expr | position}, ... [WITH ROLLUP]]
    [HAVING where_condition]
    [WINDOW window_name AS (window_spec)
        [, window_name AS (window_spec)] ...]
    [ORDER BY {col_name | expr | position}
        [ASC | DESC], ... [WITH ROLLUP]]
    [LIMIT {[offset,] row_count | row_count OFFSET offset}]
    [into_option]
    [FOR {UPDATE | SHARE}
        [OF tbl_name [, tbl_name] ...]
        [NOWAIT | SKIP LOCKED]
        | LOCK IN SHARE MODE]
    [into_option]

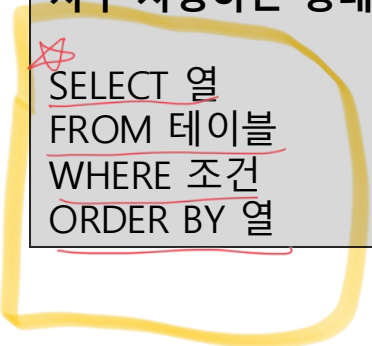
```

04-1

SELECT 문으로 데이터 조회하기

- SELECT 문을 가장 자주 사용하는 형태로 줄이면 다음과 같음

자주 사용하는 형태의 SELECT 문



SELECT 열
FROM 테이블
WHERE 조건
ORDER BY 열

▪ SELECT 문으로 열 조회하기

- 데이터를 조회하려면 SELECT 문을 사용함
- SELECT 문은 다양한 옵션도 함께 사용할 수 있음

SELECT 문의 기본 형식

SELECT [열] FROM [테이블]

① ② ③ ④

① SELECT: 데이터를 조회하겠다는 구문이다.

② [열]: 조회하고 싶은 데이터의 열을 입력한다. 여러 열을 조회할 때에는 쉼표로 구분하여 연결한다. 열 이름 대신 *를 입력하면 테이블의 전체 열을 조회할 수 있다.

③ FROM: 데이터를 가져올 테이블을 정하는 구문이다.

④ [테이블]: 데이터가 저장되어 있는 테이블 이름을 입력한다. 이때 DB.테이블명과 같이 DB명을 함께 붙여서 사용할 경우 해당 데이터베이스의 테이블을 조회할 수 있다.

▪ SELECT 문으로 열 조회하기

- 데이터를 조회하기 전에 customer 테이블의 구조를 살펴보자
- sakila 데이터베이스의 customer 테이블은 모든 고객의 정보 데이터가 저장되어 있음

customer 테이블	
customer_id	이 테이블에서 고객을 고유하게 식별하는 데 사용하는 기본키
store_id	store 테이블에서 고객의 '홈 스토어'를 식별하는 외래키
first_name	고객의 이름이 저장되어 있는 열
last_name	고객의 성이 저장되어 있는 열
email	고객의 이메일 주소가 저장되어 있는 열
address_id	address 테이블에서 고객의 주소를 식별하는 외래키
active	고객이 활성화된 고객인지 여부가 저장되어 있는 열
create_date	고객이 시스템에 추가된 날짜가 저장되어 있는 열
last_update	행이 수정되었거나 가장 최근에 업데이트된 시간이 저장되어 있는 열

▪ SELECT 문으로 열 조회하기

▪ 하나의 열 조회하기

- customer 테이블에서 고객의 이름이 저장된 first_name 열을 조회하는 쿼리를 작성해 보자

Do it! first_name 열을 조회

```
SELECT first_name FROM customer;
```

실행 결과

	first_name
▶	MARY
	PATRICIA
	LINDA
	BARBARA
	ELIZABETH
	JENNIFER
	MARIA
	SUSAN
	MARGARET
	DOROTHY

▪ SELECT 문으로 열 조회하기

▪ 2개 이상의 열 조회하기

- 이번에는 2개의 열을 조회하는 쿼리를 알아보자
- 다음은 customer 테이블에서 first_name, last_name 열을 조회하는 쿼리임
- 이때, 2개의 열을 조회하기 위해 쉼표(,)로 조회할 열 이름을 나열함

Do it! 2개의 열을 조회

```
SELECT first_name, last_name FROM  
customer;
```

실행 결과

	first_name	last_name
▶	MARY	SMITH
	PATRICIA	JOHNSON
	LINDA	WILLIAMS
	BARBARA	JONES
	ELIZABETH	BROWN
	JENNIFER	DAVIS
	MARIA	MILLER
	SUSAN	WILSON
	MARGARET	MOORE
	DOROTHY	TAYLOR

SELECT 문으로 열 조회하기

전체 열 조회하기

- 테이블의 전체 열을 조회하려면 열 이름을 모두 입력할 필요 없이 * 기호를 사용하면 됨
- 선택표로 구분하여 모든 열을 입력해도 되지만
열 이름을 외우기 어려우니 * 기호를 사용하는 것을 추천함
- 다음은 customer 테이블의 전체 데이터를 조회하는 쿼리임

Do it! 모든 열을 조회

```
SELECT * FROM  
customer;
```

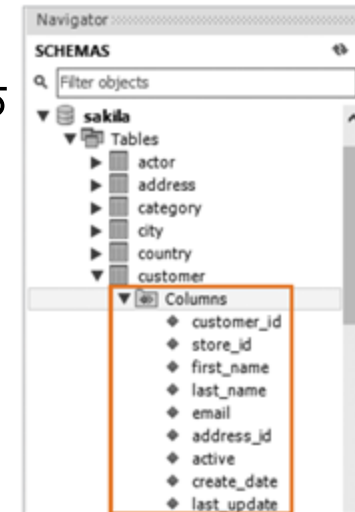
전체 조회

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	5	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	6	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	10	1	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org	14	1	2006-02-14 22:04:36	2006-02-15 04:57:20

■ MySQL 워크벤치에서 테이블의 열 정보 확인하기

- 데이터를 조회할 때 테이블에 무슨 열이 있는지 확인하고 싶은 경우
왼쪽 내비게이터의 [Schemas] 탭에서 테이블을 확장하면
열의 정보를 쉽게 확인할 수 있음
- 예를 들어 오른쪽과 같이
[sakila → Tables → customer → Columns] 순서로 확장하
현재 사용하는 테이블의 열 정보를 확인할 수 있음



- MySQL 워크벤치에서 테이블의 열 정보 확인하기
 - 또는 MySQL 서버에서 제공하는 시스템 함수를 활용해 테이블 열 정보를 확인할 수도 있음
 - 이러한 시스템 함수는 데이터베이스 시스템마다 지원하는 명령어가 다름
 - 여기에서는 MySQL에서만 적용되는 SHOW COLUMNS를 사용해 보자

MySQL 워크벤치에서 테이블의 열 정보 확인하기

- 다음은 customer 테이블의 열 정보를 확인하는 쿼리임

Do it! customer 테이블의 열 정보를 조회

```
SHOW          COLUMNS          FROM
sakila.customer;
```

실행 결과

Field	Type	Null	Key	Default	Extra
customer_id	smallint unsigned	NO	PRI	NULL	auto_increment
store_id	tinyint unsigned	NO	MUL	NULL	
first_name	varchar(45)	NO		NULL	
last_name	varchar(45)	NO	MUL	NULL	
email	varchar(50)	YES		NULL	
address_id	smallint unsigned	NO	MUL	NULL	
active	tinyint(1)	NO		1	
create_date	datetime	NO		NULL	
last_update	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT_TI...

- 테이블에 생성되어 있는 열 이름이 출력되고,
각 열의 데이터 유형이나 Null 값 허용 여부 등의 정보를 확인할 수 있음

- 시스템의 과부하를 줄이려면 필요한 데이터만 조회해야 함
- 지금까지 필요한 열을 조회하는 방법은 알아보았지만
행은 전체가 다 조회되므로
아직 필요한 데이터만 조회했다고 볼 수 없음
- 실무에서 사용하는 데이터베이스에는
엄청난 양의 데이터가 저장되어 있으므로
매번 전체 행을 가져오면 안 됨
- 이번에는 조건에 맞는 행을 조회하기 위해
WHERE 문을 활용하는 방법을 알아보자

- WHERE 문의 기본 형식

WHERE 문의 기본 형식

```
SELECT [열] FROM [테이블] WHERE [열] = [조건값]
```

① ② ③ ④

- ① WHERE: 조건에 맞는 행을 조회하기 위한 구문이다.
- ② [열]: 조건을 적용할 열을 입력한다.
- ③ 연산자: 조건을 적용할 연산자 종류를 입력한다. 여기서는 =을 입력했다.
- ④ [조건값]: 사용자 조건값을 입력한다.

▪ WHERE 문으로 특정 값 조회하기

- 실습을 통해서 WHERE 문을 어떻게 사용하는지 알아보자
- 다음은 customer 테이블에서 first_name 열의 값이 'MARIA'인 데이터를 조회한 쿼리임

Do it! WHERE 문, = 연산자로 특정 값 조회

```
SELECT * FROM customer WHERE first_name =  
'MARIA';
```

WHERE 문으로 특정 값 조회하기

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- = 연산자 오른쪽 조건에 해당하는 데이터만 조회된 것을 확인할 수 있음
- 이렇게 필요한 데이터만 필터링하면 원하는 데이터를 빠르게 조회할 수 있고 시스템의 부하를 줄일 수도 있음

- WHERE 문에서 비교 연산자 사용해 보기

- WHERE 문에는 다양한 연산자를 사용할 수 있음
- 앞선 예에서는 조건에 해당하는 값과 같은 데이터를 조회하기 위해 비교 연산자 =를 사용함
- 이외에도 MySQL은 다양한 비교 연산자를 제공함

WHERE 문에서 비교 연산자 사용해 보기

비교 연산자의 종류 살펴보기

- MySQL에서는 다음과 같은 비교 연산자를 사용하여 데이터를 조회할 수 있음

연산자	설명
<	조건값보다 작은 값을 조회한다.
<=	조건값보다 같거나 작은 값을 조회한다.
=	조건값과 동일한 값을 조회한다.
>	조건값보다 큰 값을 조회한다.
>=	조건값보다 같거나 큰 값을 조회한다.
<>, !=	조건값과 동일하지 않는 값을 조회한다.
!<	조건값보다 작지 않은 값을 조회한다.
!>	조건값보다 크지 않은 값을 조회한다.

- <= 등과 같이 크기를 비교하는 연산자는 숫자에만 사용하기를 권함
- 문자열에도 비교 연산자를 사용할 수 있지만 이 연산자는 데이터베이스 시스템이 정의한 정렬 기준에 따라 결과값이 달라지므로 데이터 속성에 따라 적절히 사용해야 함

WHERE 문에서 비교 연산자 사용해 보기

비교 연산자를 이용한 데이터 조회하기

- 숫자형, 문자열형, 날짜형 등 여러 데이터 유형과 비교 연산자를 사용했을 때 어떤 결과가 나오는지 여러 예를 통해 살펴보자

- customer 테이블에서 숫자형 비교로 address_id 열의 데이터가 200에 해당하는 행만 조회해 보자

Do it! address_id가 200인 행을 조회

```
SELECT * FROM customer WHERE address_id = 200;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	196	1	ALMA	AUSTIN	ALMA.AUSTIN@sakilacustomer.org	200	1	2006-02-14 22:04:36	2006-02-15 04:57:20

WHERE 문에서 비교 연산자 사용해 보기

비교 연산자를 이용한 데이터 조회하기

- 마찬가지로 customer 테이블에서 이번에는 숫자형 비교로 address_id 열에서 데이터가 200 미만에 해당하는 행을 조회해 보자

Do it! address_id가 200 미만인 행을 조회

```
SELECT * FROM customer WHERE address_id < 200;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	5	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	6	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	10	1	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org	14	1	2006-02-14 22:04:36	2006-02-15 04:57:20

WHERE 문으로 조건에 맞는 데이터 조회하기

- **비교 연산자를 이용한 데이터 조회하기**

3. 다음은 customer 테이블에서 문자열형 비교로 first_name 열에서 데이터가 MARIA인 행을 조회해 보자

Do it! first_name⁰ | MARIA인 행을 조회

```
SELECT * FROM customer WHERE first_name = 'MARIA';
```

실행 결과

[illegible]

WHERE 문에서 비교 연산자 사용해 보기

비교 연산자를 이용한 데이터 조회하기

4. customer 테이블에서 문자열형 비교로 first_name 열에서 데이터가 A, B, C 순으로 MARIA보다 앞에 위치한 행들을 조회해 보자

Do it! first_name이 MARIA 미만인 행을 조회

```
SELECT * FROM customer WHERE first_name < 'MARIA';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	10	1	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org	14	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	11	2	LISA	ANDERSON	LISA.ANDERSON@sakilacustomer.org	15	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	13	2	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org	17	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	14	2	BETTY	WHITE	BETTY.WHITE@sakilacustomer.org	18	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	15	1	HELEN	HARRIS	HELEN.HARRIS@sakilacustomer.org	19	1	2006-02-14 22:04:36	2006-02-15 04:57:20

WHERE 문으로 조건에 맞는 데이터 조회하기

- **비교 연산자를 이용한 데이터 조회하기**

Do it! payment_date가 2005-07-09 13:24:07인 행을 조회

실행 결과

[illegible]

WHERE 문에서 비교 연산자 사용해 보기

비교 연산자를 이용한 데이터 조회하기

6. 마지막으로 payment 테이블에서 날짜형 비교로 payment_date 열에서 데이터가 2005년 7월 9일 이전 날짜에 해당하는 행들을 조회해 보자

Do it! payment_date가 2005년 7월 9일 미만인 행을 조회

```
SELECT * FROM payment  
WHERE payment_date < '2005-07-09';
```

실행 결과

	payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
▶	1	1	1	76	2.99	2005-05-25 11:30:37	2006-02-15 22:12:30
	2	1	1	573	0.99	2005-05-28 10:35:23	2006-02-15 22:12:30
	3	1	1	1185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
	4	1	2	1422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
	5	1	2	1476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30
	6	1	1	1725	4.99	2005-06-16 15:18:57	2006-02-15 22:12:30
	7	1	1	2308	4.99	2005-06-18 08:41:48	2006-02-15 22:12:30
	8	1	2	2363	0.99	2005-06-18 13:33:59	2006-02-15 22:12:30
	9	1	1	3284	3.99	2005-06-21 06:24:45	2006-02-15 22:12:30
	10	1	2	4526	5.99	2005-07-08 03:17:05	2006-02-15 22:12:30

WHERE 문에서 논리 연산자 사용해 보기

논리 연산자의 종류 살펴보기

- 논리 연산자를 표로 정리함
- 이와 같은 논리 연산자를 사용하여 다양한 조건을 주어 데이터를 조회할 수 있음

연산자	설명
ALL	모든 비교 집합이 참일 경우에 해당 데이터를 조회한다.
AND	AND를 기준으로 양쪽의 부울 표현식이 모두 참이면 해당 데이터를 조회한다.
ANY	비교 집합 중 하나라도 참이면 해당 데이터를 조회한다.
BETWEEN	피연산자가 범위 내에 있으면 데이터를 조회한다.
EXISTS	하위 쿼리에 행이 포함되면 데이터를 조회한다.
IN	피연산자가 리스트 중 하나라도 포함되어 있으면 데이터를 조회한다.
LIKE	피연산자가 패턴과 일치하는 경우 데이터를 조회한다.
NOT	부울 연산자를 반대로 실행해 데이터를 조회한다.
OR	OR를 기준으로 한 쪽의 부울 표현식이 참이면 해당 데이터를 조회한다.
SOME	비교 집합 중 일부가 참인 경우 데이터를 조회한다.

- WHERE 문에서 논리 연산자 사용해 보기

- BETWEEN을 이용한 데이터 조회하기

- BETWEEN은 WHERE 문과 함께 사용하며 조회하는 값의 범위를 지정할 수 있음
 - BETWEEN은 보통 날짜형 데이터를 조회할 때 자주 사용하지만 문자열형, 숫자형 데이터에도 사용할 수 있음

- WHERE 문에서 논리 연산자 사용해 보기

- BETWEEN을 이용한 데이터 조회하기

- 다음 쿼리는 customer 테이블의 address_id 열에서 데이터가 5~10 범위에 해당하는 행을 조회함

Do it! 정해진 범위에 해당하는 데이터 조회

```
SELECT * FROM customer WHERE address_id BETWEEN 5 AND 10;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	5	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	6	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- WHERE 문에서 논리 연산자 사용해 보기

- BETWEEN을 이용한 데이터 조회하기

1. 조회 결과를 살펴보면

address_id 열에서 5, 6, 7, 8, 9, 10에 해당하는 데이터가 반환된 것을 확인할 수 있음.

이를 통해 BETWEEN은 조건값을 포함하여 데이터를 조회한다는 것을 알 수 있음

- WHERE 문에서 논리 연산자 사용해 보기

- BETWEEN을 이용한 데이터 조회하기

2. 이번에는 날짜형 데이터를 조회할 때에도 BETWEEN을 사용해 보자.

다음 쿼리는 payment 테이블의 payment_date 열에서
데이터가 2005년 6월 17일~2005년 7월 19일 범위에 해당하는 행을 조회함.

이 역시도 조건값을 포함한 결과를 반환함

Do it! 2005년 6월 17일~2005년 7월 19일을 포함한 날짜 조회

```
SELECT * FROM payment WHERE payment_date BETWEEN '2005-06-17' AND '2005-07-19';
```

- WHERE 문에서 논리 연산자 사용해 보기
 - BETWEEN을 이용한 데이터 조회하기

2.

실행 결과							
	payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
▶	7	1	1	2308	4.99	2005-06-18 08:41:48	2006-02-15 22:12:30
	8	1	2	2363	0.99	2005-06-18 13:33:59	2006-02-15 22:12:30
	9	1	1	3284	3.99	2005-06-21 06:24:45	2006-02-15 22:12:30
	10	1	2	4526	5.99	2005-07-08 03:17:05	2006-02-15 22:12:30
	11	1	1	4611	5.99	2005-07-08 07:33:56	2006-02-15 22:12:30
	12	1	1	5244	4.99	2005-07-09 13:24:07	2006-02-15 22:12:30
	13	1	1	5326	4.99	2005-07-09 16:38:01	2006-02-15 22:12:30
	14	1	1	6163	7.99	2005-07-11 10:13:46	2006-02-15 22:12:30
	34	2	1	2128	2.99	2005-06-17 20:54:58	2006-02-15 22:12:30
	35	2	1	5636	2.99	2005-07-10 06:31:24	2006-02-15 22:12:30

날짜형 데이터를 조회할 때는 밀리초를 고려해야 함.
예를 들어 2005-06-17을 조건값으로 사용하면
실제로는 2005-06-17 00:00:00.000이 조건식에 사용됨.
때문에 이와 같은 결과가 출력됨

- WHERE 문에서 논리 연산자 사용해 보기

- BETWEEN을 이용한 데이터 조회하기

- 시간을 포함해 정확한 날짜형 데이터를 조건값으로 사용하려면 다음과 같이 쿼리를 작성하면 됨

Do it! 정확한 날짜를 조회

```
SELECT * FROM payment WHERE payment_date = '2005-07-08 07:33:56';
```

실행 결과

	payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
▶	11	1	1	4611	5.99	2005-07-08 07:33:56	2006-02-15 22:12:30

WHERE 문에서 논리 연산자 사용해 보기

BETWEEN을 이용한 데이터 조회하기

4. 다음 쿼리는 문자열 범위를 지정하여 데이터를 조회함.
Customer 테이블의 first_name 열에서
M과 O 사이의 데이터를 조회한 뒤, 결과를 확인해 보자

Do it! first_name 열에서 M~O 범위의 데이터 조회

```
SELECT * FROM customer
WHERE first_name BETWEEN 'M' AND 'O';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	5	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	12	1	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org	16	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	21	1	MICHELLE	CLARK	MICHELLE.CLARK@sakilacustomer.org	25	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	30	1	MELISSA	KING	MELISSA.KING@sakilacustomer.org	34	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	38	1	MARTHA	GONZALEZ	MARTHA.GONZALEZ@sakilacustomer.org	42	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	44	1	MARIE	TURNER	MARIE.TURNER@sakilacustomer.org	48	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	60	1	MILDRED	BAILEY	MILDRED.BAILEY@sakilacustomer.org	64	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	68	1	NICOLE	PETERSON	NICOLE.PETERSON@sakilacustomer.org	72	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	80	1	MARILYN	ROSS	MARILYN.ROSS@sakilacustomer.org	84	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	94	1	NORMA	GONZALES	NORMA.GONZALES@sakilacustomer.org	98	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	128	1	MARJORIE	TUCKER	MARJORIE.TUCKER@sakilacustomer.org	132	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	131	2	MONICA	HICKS	MONICA.HICKS@sakilacustomer.org	135	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	151	2	MEGAN	PALMER	MEGAN.PALMER@sakilacustomer.org	155	1	2006-02-14 22:04:36	2006-02-15 04:57:20

WHERE 문에서 논리 연산자 사용해 보기

BETWEEN을 이용한 데이터 조회하기

5. 반대로 BETWEEN을 사용하되 범위를 포함하지 않은 데이터를 조회해 보자.
이때는 간단히 NOT 연산자를 조합하면 됨

Do it! first_name 열에서 M~O 범위의 값을 제외한 데이터 조회

```
SELECT * FROM customer  
WHERE first_name NOT BETWEEN 'M' AND 'O';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	6	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

- 지금까지는 WHERE 문에 값으로만 조건을 설정해 데이터를 조회함
 - 하지만 현실에서는 다양하고 복잡한 조건을 적용해야 하는 경우가 많은데, 그럴 때는 논리 연산자인 AND와 OR를 사용하여 값뿐만 아니라 식을 활용한 여러 조건을 결합할 수 있음
 - 논리 연산자 AND는 두 개의 조건이 모두 만족할 때 조건에 해당하는 데이터만 조회하고,

OR는 입력한 두 조건 중 하나라도 만족할 때 조건에 해당하는 데이터를 모두 조회함

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

1. 다음은 city 테이블에서 city 열이 'Sunnyvale'이면서 country_id 열이 103인 데이터를 조회하는 쿼리임

Do it! 두 조건을 만족하는 데이터 조회

```
SELECT * FROM city WHERE city = 'Sunnyvale' AND country_id= 103;
```

실행 결과

	city_id	city	country_id	last_update
▶	508	Sunnyvale	103	2006-02-15 04:45:25

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

- 2. 앞서 BETWEEN으로 날짜 범위를 지정할 때 AND 연산자를 활용함.

- 이번에는 WHERE 문에 비교 연산자를 활용한 두 개의 조건과 AND 연산자를 이용해 payment 테이블에서 지정한 날짜 범위만큼의 데이터를 조회함

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

2.

Do it! 두 개의 비교 연산식을 만족하는 데이터 조회

```
SELECT * FROM payment  
WHERE payment_date >= '2005-06-01' AND payment_date <= '2005-07-05';
```

실행 결과

	payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
▶	3	1	1	1185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
	4	1	2	1422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
	5	1	2	1476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30
	6	1	1	1725	4.99	2005-06-16 15:18:57	2006-02-15 22:12:30
	7	1	1	2308	4.99	2005-06-18 08:41:48	2006-02-15 22:12:30
	8	1	2	2363	0.99	2005-06-18 13:33:59	2006-02-15 22:12:30
	9	1	1	3284	3.99	2005-06-21 06:24:45	2006-02-15 22:12:30
	34	2	1	2128	2.99	2005-06-17 20:54:58	2006-02-15 22:12:30
	62	3	1	1546	8.99	2005-06-16 01:34:05	2006-02-15 22:12:30
	63	3	1	1726	6.99	2005-06-16 15:19:10	2006-02-15 22:12:30

WHERE 문에서 논리 연산자 사용해 보기

AND와 OR를 이용한 데이터 조회하기

3. AND와 달리 OR는 양쪽 조건 중 하나라도 만족하면 결과를 모두 반환함.
다음은 customer 테이블의 first_name 열에서
'MARIA' 또는 'LINDA'인 데이터를 조회하는 쿼리임

Do it! 한 조건을 만족한 경우 데이터 조회

```
SELECT * FROM customer  
WHERE first_name = 'MARIA' OR first_name = 'LINDA';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20

WHERE 문에서 논리 연산자 사용해 보기

AND와 OR를 이용한 데이터 조회하기

4. 여러 데이터를 조회하기 위해 OR를 여러 번 사용하면 동일한 연산자를 반복해서 작성하므로 비효율적임.
다음은 쿼리를 통해 살펴보자

Do it! OR를 두 개 이상 사용한 경우

```
SELECT * FROM customer  
WHERE first_name = 'MARIA' OR first_name = 'LINDA' OR first_name = 'NANCY';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	12	1	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org	16	1	2006-02-14 22:04:36	2006-02-15 04:57:20

WHERE 문에서 논리 연산자 사용해 보기

AND와 OR를 이용한 데이터 조회하기

4. 이런 경우에는 OR를 반복하여 쓰기 보단 IN을 사용해 보자.
IN은 OR와 같은 역할을 하지만 연산자를 한 개만 작성하고, 쉼표를 사용해 조건을 나열함.
IN을 활용해 앞에서 작성한 쿼리를 수정하면 결과는 다음과 같음

Do it! IN을 활용한 데이터 조회

```
SELECT * FROM customer  
WHERE first_name IN ('MARIA', 'LINDA','NANCY');
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	12	1	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org	16	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

5. 이번에는 AND, OR, IN을 조합하여 데이터를 조회해 보자.
이번에는 조회해야 할 데이터 조건(요구 사항)을 살펴보고 이를 조회해 보자

요구 사항

city 테이블에서 country_id 열이 103 또는 country_id 열이 86이면서 city 열이 'Cheju', 'Sunnyvale', 'Dallas'인 데이터를 조회

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

5. 이 요구 사항을 보고 쿼리를 그냥 작성했다면 아마 다음과 같을 것임

Do it! 요구 사항을 반영해 작성한 쿼리

```
SELECT * FROM city  
WHERE country_id = 103 OR country_id  
= 86  
AND city IN ('Cheju', 'Sunnyvale', 'Dallas');
```

실행 결과

	city_id	city	country_id	last_update
▶	113	Cheju	86	2006-02-15 04:45:25
	11	Akron	103	2006-02-15 04:45:25
	33	Arlington	103	2006-02-15 04:45:25
	41	Augusta-Richmond County	103	2006-02-15 04:45:25
	42	Aurora	103	2006-02-15 04:45:25
	65	Bellevue	103	2006-02-15 04:45:25
	94	Brockton	103	2006-02-15 04:45:25
	101	Cape Coral	103	2006-02-15 04:45:25
	120	Citrus Heights	103	2006-02-15 04:45:25
	123	Clarksville	103	2006-02-15 04:45:25

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

- 요구 사항대로라면 3개의 행으로 구성된 데이터가 조회돼야 하는데 결과는 그렇지 않음.
그 이유는 논리 연산자의 우선순위 때문.
앞서 작성한 쿼리는 사실 다음과 같은 순서로 풀이됨

쿼리 풀이 순서 1

```
SELECT * FROM city WHERE country_id = 103;
```

쿼리 풀이 순서 2

```
SELECT * FROM city  
WHERE country_id = 86  
      AND city IN ('Cheju', 'Sunnyvale', 'Dallas');
```

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

5. 위와 같은 순서로 풀이되었기 때문에
country_id 열이 103인 전체 데이터와
country_id 열이 86인 전체 데이터 중에서
city 열이 'Cheju', 'Sunnyvale', 'Dallas' 데이터가 조회됨.

만약 country_id가 103인 쿼리문과 country_id가 86인 쿼리문의 순서를 바꾸면
논리 연산자 우선순위 때문에 또 다른 결과가 발생하게 됨

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

5.

Do it! 쿼리 순서를 변경

```
SELECT * FROM city
WHERE country_id = 86 OR country_id = 103
      AND city IN ('Cheju', 'Sunnyvale', 'Dallas' );
```

실행 결과

	city_id	city	country_id	last_update
▶	113	Cheju	86	2006-02-15 04:45:25
	268	Kimchon	86	2006-02-15 04:45:25
	357	Naju	86	2006-02-15 04:45:25
	539	Tonghae	86	2006-02-15 04:45:25
	553	Uijongbu	86	2006-02-15 04:45:25
	135	Dallas	103	2006-02-15 04:45:25
	508	Sunnyvale	103	2006-02-15 04:45:25

WHERE 문에서 논리 연산자 사용해 보기

AND와 OR를 이용한 데이터 조회하기

5. 원하는 대로 결과를 도출하려면 쿼리를 수정할 때 소괄호를 사용하면 됨.
소괄호로 먼저 실행되어야 하는 쿼리에 우선순위를 정해 줄 수 있음.

다음 쿼리는 country_id 열과 관련된 조건에 대한 결과를 먼저 도출한 뒤,
city 열 조건을 필터링하여 진짜로 원했던 데이터 3개를 반환함

Do it! 소괄호로 우선순위를 다시 정해 데이터를 조회

```
SELECT * FROM city
WHERE (country_id = 103 OR country_id = 86)
      AND city IN ('Cheju', 'Sunnyvale',
                  'Dallas');
```

실행 결과

	city_id	city	country_id	last_update
▶	113	Cheju	86	2006-02-15 04:45:25
	268	Kimchon	86	2006-02-15 04:45:25
	357	Naju	86	2006-02-15 04:45:25
	539	Tonghae	86	2006-02-15 04:45:25
	553	Uijongbu	86	2006-02-15 04:45:25
	135	Dallas	103	2006-02-15 04:45:25
	508	Sunnyvale	103	2006-02-15 04:45:25

- WHERE 문에서 논리 연산자 사용해 보기

- AND와 OR를 이용한 데이터 조회하기

- 또는 country_id 열과 관련된 조건도 IN으로 묶은 다음, 쿼리를 작성해도 앞선 실습과 같은 결과를 얻을 수 있음

Do it! IN, AND를 결합하여 조회

```
SELECT * FROM city
WHERE (country_id = 103 OR country_id = 86)
      AND city IN ('Cheju', 'Sunnyvale',
                  'Dallas');
```

실행 결과

	city_id	city	country_id	last_update
▶	113	Cheju	86	2006-02-15 04:45:25
	135	Dallas	103	2006-02-15 04:45:25
	508	Sunnyvale	103	2006-02-15 04:45:25

- WHERE 문에서 논리 연산자 사용해 보기

- NULL 데이터 조회하기

- 데이터베이스에서 데이터를 조회하다 보면 NULL값이 포함된 열을 자주 볼 수 있음.

NULL이란 데이터가 없는 상태를 말함.

더 풀어 설명하자면 숫자 0, 공백이 아니라 **아예 정의되지 않은 값**을 말함

WHERE 문에서 논리 연산자 사용해 보기

NULL 데이터 조회하기

1. address 테이블을 조회해 보자.
address2 열을 살펴보면 공백으로 표시된 곳과 NULL이 표시된 곳이 구분되어 있음

Do it! Null이 있는 테이블 조회

```
SELECT * FROM address;
```

실행 결과

address_id	address	address2	district	city_id	postal_code	phone	location	last_update
1	47 MySakila Drive	NULL	Alberta	300			BL0B	2014-09-25 22:30:27
2	28 MySQL Boulevard	NULL	QLD	576			BL0B	2014-09-25 22:30:09
3	23 Workhaven Lane	NULL	Alberta	300		14033335568	BL0B	2014-09-25 22:30:27
4	1411 Lillydale Drive	NULL	QLD	576		6172235589	BL0B	2014-09-25 22:30:09
5	1913 Hanoi Way		Nagasaki	463	35200	28303384290	BL0B	2014-09-25 22:31:53
6	1121 Loja Avenue		California	449	17886	838635286649	BL0B	2014-09-25 22:34:01
7	692 Joliet Street		Attika	38	83579	448477190408	BL0B	2014-09-25 22:31:07
8	1566 Inegl Manor		Mandalay	349	53561	705814003527	BL0B	2014-09-25 22:32:18
9	53 Idfu Parkway		Nantou	361	42399	10655648674	BL0B	2014-09-25 22:33:16
10	1795 Santiago de Compostela Way		Texas	295	18743	860452626434	BL0B	2014-09-25 22:33:55

▪ WHERE 문에서 논리 연산자 사용해 보기

▪ NULL 데이터 조회하기

2. NULL은 IS NULL, IS NOT NULL을 사용하여 필터링함.
'= 연산자로 찾을 수 있지 않을까?'라고 생각이 들 수 있으므로
= 연산자를 사용하여 NULL을 찾는 쿼리를 작성하여 실행해 보자

Do it! = 연산자를 사용해 NULL 데이터 조회

```
SELECT * FROM address WHERE address2 = NULL;
```

실행 결과

	address_id	address	address2	district	city_id	postal_code	phone	location	last_update
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

NULL은 정의되지 않은 값이므로 일반적인 연산자로 조회할 수 없어 아무것도 얻지 못함

- WHERE 문에서 논리 연산자 사용해 보기

- NULL 데이터 조회하기

3. 이번에는 IS NULL을 사용해 NULL 데이터를 조회해 보자

Do it! address2 열에서 NULL인 데이터 조회

```
SELECT * FROM address WHERE address2 IS NULL;
```

실행 결과

	address_id	address	address2	district	city_id	postal_code	phone	location	last_update
▶	1	47 MySakila Drive	NULL	Alberta	300			BLOB	2014-09-25 22:30:27
	2	28 MySQL Boulevard	NULL	QLD	576			BLOB	2014-09-25 22:30:09
	3	23 Workhaven Lane	NULL	Alberta	300		14033335568	BLOB	2014-09-25 22:30:27
	4	1411 Lillydale Drive	NULL	QLD	576		6172235589	BLOB	2014-09-25 22:30:09

WHERE 문에서 논리 연산자 사용해 보기

NULL 데이터 조회하기

4. 반대로 NULL이 아닌 데이터를 조회할 때에도
일반 연산자가 아닌 IS NOT NULL을 사용해 데이터를 조회함

Do it! address2 열에서 NULL이 아닌 데이터 조회

```
SELECT * FROM address WHERE address2 IS NOT NULL;
```

실행 결과

	address_id	address	address2	district	city_id	postal_code	phone	location	last_update
▶	5	1913 Hanoi Way		Nagasaki	463	35200	28303384290	BLOB	2014-09-25 22:31:53
	6	1121 Loja Avenue		California	449	17886	838635286649	BLOB	2014-09-25 22:34:01
	7	692 Joliet Street		Attika	38	83579	448477190408	BLOB	2014-09-25 22:31:07
	8	1566 Inegl Manor		Mandalay	349	53561	705814003527	BLOB	2014-09-25 22:32:18
	9	53 Idfu Parkway		Nantou	361	42399	10655648674	BLOB	2014-09-25 22:33:16
	10	1795 Santiago de Compostela Way		Texas	295	18743	860452626434	BLOB	2014-09-25 22:33:55
	11	900 Santiago de Compostela Parkway		Central Serbia	280	93896	716571220373	BLOB	2014-09-25 22:34:11
	12	478 Joliet Way		Hamilton	200	77948	657282285970	BLOB	2014-09-25 22:32:22
	13	613 Korolev Drive		Masqat	329	45844	380657522649	BLOB	2014-09-25 22:32:29
	14	1531 Sal Drive		Esfahan	162	53628	648856936185	BLOB	2014-09-25 22:31:36

- WHERE 문에서 논리 연산자 사용해 보기

- NULL 데이터 조회하기

- 5. 이번엔 address2에 데이터가 공백인 행을 조회해 보자.

- 공백은 NULL이 아니기 때문에
다음과 같이 연산자 =으로 조회할 수 있음.

- 이때, 공백은 ' '으로 표현함

04-2

WHERE 문으로 조건에 맞는 데이터 조회하기

- WHERE 문에서 논리 연산자 사용해 보기
 - NULL 데이터 조회하기

5. Do it! address2 열에서 NULL이 아닌 데이터 조회

```
SELECT * FROM address WHERE address2 = '';
```

실행 결과

	address_id	address	address2	district	city_id	postal_code	phone	location	last_update
▶	5	1913 Hanoi Way		Nagasaki	463	35200	28303384290	BL0E	2014-09-25 22:31:53
	6	1121 Loja Avenue		California	449	17886	838635286649	BL0E	2014-09-25 22:34:01
	7	692 Joliet Street		Attika	38	83579	448477190408	BL0E	2014-09-25 22:31:07
	8	1566 Inegl Manor		Mandalay	349	53561	705814003527	BL0E	2014-09-25 22:32:18
	9	53 Idfu Parkway		Nantou	361	42399	10655648674	BL0E	2014-09-25 22:33:16
	10	1795 Santiago de Compostela Way		Texas	295	18743	860452626434	BL0E	2014-09-25 22:33:55
	11	900 Santiago de Compostela Parkway		Central Serbia	280	93896	716571220373	BL0E	2014-09-25 22:34:11
	12	478 Joliet Way		Hamilton	200	77948	657282285970	BL0E	2014-09-25 22:32:22
	13	613 Korolev Drive		Masqat	329	45844	380657522649	BL0E	2014-09-25 22:32:29
	14	1531 Sal Drive		Esfahan	162	53628	648856936185	BL0E	2014-09-25 22:31:36

04-3

ORDER BY 문으로 데이터 정렬하기

- 실습을 진행하다 보면 여러분의 결과가 다르게 보일 수 있음
- 이는 정렬 순서에 따라 결과가 다른 것인데, 정렬이 중요한 데이터를 활용해야 할 경우라면 ORDER BY로 하면 해결할 수 있음
- 조회한 데이터를 정렬하기 위해서는 ORDER BY 문을 다음과 같이 SELECT 문의 가장 마지막에 추가하여 사용하면 됨

ORDER BY 문의 기본 형식

```
SELECT [열] FROM [테이블] WHERE [열] = [조건값] ORDER BY [열] [ASC 또는 DESC]
```

① ② ③

① ORDER BY: 조회한 데이터를 정렬하기 위한 구문이다.

② [열]: 정렬할 열 이름을 입력한다.

③ [ASC 또는 DESC]: 정렬 기준을 정하기 위해 오름차순(ASC) 또는 내림차순(DESC)을 명시한다.

▪ ORDER BY 문으로 열 기준 정렬하기

- 데이터를 정렬할 때 정렬할 열 이름을 ORDER BY 문 뒤에 입력함

1. 다음은 각각 first_name 열과 last_name 열 기준으로 정렬하는 쿼리임

Do it! first_name 열을 기준으로 정렬

```
SELECT * FROM customer ORDER BY first_name;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	375	2	AARON	SELBY	AARON.SELBY@sakilacustomer.org	380	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	367	1	ADAM	GOOCH	ADAM.GOOCH@sakilacustomer.org	372	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	525	2	ADRIAN	CLARY	ADRIAN.CLARY@sakilacustomer.org	531	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	217	2	AGNES	BISHOP	AGNES.BISHOP@sakilacustomer.org	221	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	389	1	ALAN	KAHN	ALAN.KAHN@sakilacustomer.org	394	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	352	1	ALBERT	CROUSE	ALBERT.CROUSE@sakilacustomer.org	357	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	568	2	ALBERTO	HENNING	ALBERTO.HENNING@sakilacustomer.org	574	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	454	2	ALEX	GRESHAM	ALEX.GRESHAM@sakilacustomer.org	459	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	439	2	ALEXANDER	FENNELL	ALEXANDER.FENNELL@sakilacustomer.org	444	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	423	2	ALFRED	CASILLAS	ALFRED.CASILLAS@sakilacustomer.org	428	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	567	2	ALFREDO	MCADAMS	ALFREDO.MCADAMS@sakilacustomer.org	573	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	51	1	ALICE	STEWART	ALICE.STEWART@sakilacustomer.org	55	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	152	1	ALICIA	MILLS	ALICIA.MILLS@sakilacustomer.org	156	1	2006-02-14 22:04:36	2006-02-15 04:57:20

04-3

ORDER BY 문으로 데이터 정렬하기

▪ ORDER BY 문으로 열 기준 정렬하기

1. Do it! last_name 열을 기준으로 정렬

```
SELECT * FROM customer ORDER BY last_name;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	505	1	RAFAEL	ABNEY	RAFAEL.ABNEY@sakilacustomer.org	510	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	504	1	NATHANIEL	ADAM	NATHANIEL.ADAM@sakilacustomer.org	509	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	36	2	KATHLEEN	ADAMS	KATHLEEN.ADAMS@sakilacustomer.org	40	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	96	1	DIANA	ALEXANDER	DIANA.ALEXANDER@sakilacustomer.org	100	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	470	1	GORDON	ALLARD	GORDON.ALLARD@sakilacustomer.org	475	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	27	2	SHIRLEY	ALLEN	SHIRLEY.ALLEN@sakilacustomer.org	31	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	220	2	CHARLENE	ALVAREZ	CHARLENE.ALVAREZ@sakilacustomer.org	224	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	11	2	LISA	ANDERSON	LISA.ANDERSON@sakilacustomer.org	15	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	326	1	JOSE	ANDREW	JOSE.ANDREW@sakilacustomer.org	331	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	183	2	IDA	ANDREWS	IDA.ANDREWS@sakilacustomer.org	187	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	449	2	OSCAR	AQUINO	OSCAR.AQUINO@sakilacustomer.org	454	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	368	1	HARRY	ARCE	HARRY.ARCE@sakilacustomer.org	373	0	2006-02-14 22:04:37	2006-02-15 04:57:20
	560	1	JORDAN	ARCHULETA	JORDAN.ARCHULETA@sakilacustomer.org	566	1	2006-02-14 22:04:37	2006-02-15 04:57:20

▪ ORDER BY 문으로 열 기준 정렬하기

1. 두 결과를 비교해 보면 first_name 열을 기준으로 했을 때는 first_name 열의 데이터들이 오름차순으로 정리된 것을 확인할 수 있음.

마찬가지로 last_name 열을 기준으로 정했을 때는 last_name 열의 데이터들이 오름차순으로 정리된 것을 확인할 수 있음

▪ ORDER BY 문으로 열 기준 정렬하기

2. 2개 이상의 열을 기준으로 정렬할 때는
쉼표를 사용하여 열 이름을 나열하면 됨.

이때 열 입력 순서에 따라 정렬 우선순위가 정해지므로
열 입력 순서에 주의하자.

정렬 순서를 store_id, first_name 순으로 작성한 쿼리임.

이렇게 하면 먼저 store_id 열을 기준으로 정렬한 다음,
store_id 열에 같은 값이 있는 경우
first_name 열을 기준으로 데이터를 정렬함

ORDER BY 문으로 열 기준 정렬하기

2. **Do it!** store_id, first_name 순으로 데이터 정렬

```
SELECT * FROM customer ORDER BY store_id, first_name;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	367	1	ADAM	GOOCH	ADAM.GOOCH@sakilacustomer.org	372	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	389	1	ALAN	KAHN	ALAN.KAHN@sakilacustomer.org	394	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	352	1	ALBERT	CROUSE	ALBERT.CROUSE@sakilacustomer.org	357	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	51	1	ALICE	STEWART	ALICE.STEWART@sakilacustomer.org	55	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	152	1	ALICIA	MILLS	ALICIA.MILLS@sakilacustomer.org	156	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	548	1	ALLAN	CORNISH	ALLAN.CORNISH@sakilacustomer.org	554	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	196	1	ALMA	AUSTIN	ALMA.AUSTIN@sakilacustomer.org	200	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	139	1	AMBER	DIXON	AMBER.DIXON@sakilacustomer.org	143	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	32	1	AMY	LOPEZ	AMY.LOPEZ@sakilacustomer.org	36	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	515	1	ANDRE	RAPP	ANDRE.RAPP@sakilacustomer.org	521	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	81	1	ANDREA	HENDERS...	ANDREA.HENDERSON@sakilacustom...	85	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	503	1	ANGEL	BARCLAY	ANGEL.BARCLAY@sakilacustomer.org	508	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	48	1	ANN	EVANS	ANN.EVANS@sakilacustomer.org	52	1	2006-02-14 22:04:36	2006-02-15 04:57:20

ORDER BY 문으로 열 기준 정렬하기

- 정렬 순서를 바꿔서도 입력해 보자.
정렬 순서가 다르므로 결과도 다르게 표시됨

Do it! first_name, store_id 순으로 데이터 정렬

```
SELECT * FROM customer ORDER BY first_name, store_id;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	375	2	AARON	SELBY	AARON.SELBY@sakilacustomer.org	380	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	367	1	ADAM	GOOCH	ADAM.GOOCH@sakilacustomer.org	372	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	525	2	ADRIAN	CLARY	ADRIAN.CLARY@sakilacustomer.org	531	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	217	2	AGNES	BISHOP	AGNES.BISHOP@sakilacustomer.org	221	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	389	1	ALAN	KAHN	ALAN.KAHN@sakilacustomer.org	394	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	352	1	ALBERT	CROUSE	ALBERT.CROUSE@sakilacustomer.org	357	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	568	2	ALBERTO	HENNING	ALBERTO.HENNING@sakilacustomer.org	574	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	454	2	ALEX	GRESHAM	ALEX.GRESHAM@sakilacustomer.org	459	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	439	2	ALEXANDER	FENNELL	ALEXANDER.FENNELL@sakilacustomer.org	444	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	423	2	ALFRED	CASILLAS	ALFRED.CASILLAS@sakilacustomer.org	428	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	567	2	ALFREDO	MCADAMS	ALFREDO.MCADAMS@sakilacustomer.org	573	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	51	1	ALICE	STEWART	ALICE.STEWART@sakilacustomer.org	55	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	152	1	ALICIA	MILLS	ALICIA.MILLS@sakilacustomer.org	156	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- 오름차순 또는 내림차순으로 정렬하기

- ORDER BY 문의 기본 형식에서
열 이름 다음에 ASC 또는 DESC를 작성하면
오름차순 또는 내림차순으로 정렬한 데이터를 조회할 수 있음
- ASC는 오름차순을 의미하는 ascending의 줄임말이고,
DESC는 내림차순을 의미하는 descending의 줄임말임

- **오름차순 또는 내림차순으로 정렬하기**

- **ASC로 데이터 정렬하기**

- 오름차순으로 정렬하는 방법을 알아보자
 - 기본 형식에서도 보았듯 ASC를 정렬하려는 열 이름 뒤에 붙이면 됨
 - 다음은 customer 테이블의 first_name 열에 대해 오름차순으로 정렬하는 쿼리임
 - 결과를 보면 알파벳순으로 데이터가 정렬되었음을 알 수 있음

04-3

ORDER BY 문으로 데이터 정렬하기

- 오름차순 또는 내림차순으로 정렬하기

- ASC로 데이터 정렬하기

Do it! first_name 열을 오름차순으로 정렬

```
SELECT * FROM customer ORDER BY first_name ASC;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	375	2	AARON	SELBY	AARON.SELBY@sakilacustomer.org	380	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	367	1	ADAM	GOOCH	ADAM.GOOCH@sakilacustomer.org	372	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	525	2	ADRIAN	CLARY	ADRIAN.CLARY@sakilacustomer.org	531	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	217	2	AGNES	BISHOP	AGNES.BISHOP@sakilacustomer.org	221	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	389	1	ALAN	KAHN	ALAN.KAHN@sakilacustomer.org	394	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	352	1	ALBERT	CROUSE	ALBERT.CROUSE@sakilacustomer.org	357	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	568	2	ALBERTO	HENNING	ALBERTO.HENNING@sakilacustomer.org	574	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	454	2	ALEX	GRESHAM	ALEX.GRESHAM@sakilacustomer.org	459	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	439	2	ALEXANDER	FENNELL	ALEXANDER.FENNELL@sakilacustomer.org	444	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	423	2	ALFRED	CASILLAS	ALFRED.CASILLAS@sakilacustomer.org	428	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	567	2	ALFREDO	MCADAMS	ALFREDO.MCADAMS@sakilacustomer.org	573	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	51	1	ALICE	STEWART	ALICE.STEWART@sakilacustomer.org	55	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	152	1	ALICIA	MILLS	ALICIA.MILLS@sakilacustomer.org	156	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- **오름차순 또는 내림차순으로 정렬하기**

- **DESC로 데이터 정렬하기**

- first_name 열에 대해 내림차순으로 정렬하는 쿼리를 작성해 보자
 - 내림차순 정렬은 정렬하려는 열 이름 뒤에 DESC를 추가하면 됨
 - 결과를 보면 Z, Y, X 순(알파벳역순)으로 데이터가 정렬되었음을 알 수 있음

04-3

ORDER BY 문으로 데이터 정렬하기

- 오름차순 또는 내림차순으로 정렬하기
 - DESC로 데이터 정렬하기

Do it! first_name 열을 내림차순으로 정렬

```
SELECT * FROM customer ORDER BY first_name DESC;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	479	1	ZACHARY	HITE	ZACHARY.HITE@sakilacustomer.org	484	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	174	2	YVONNE	WATKINS	YVONNE.WATKINS@sakilacustomer.org	178	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	190	2	YOLANDA	WEAVER	YOLANDA.WEAVER@sakilacustomer.org	194	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	212	2	WILMA	RICHARDS	WILMA.RICHARDS@sakilacustomer.org	216	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	359	2	WILLIE	MARKHAM	WILLIE.MARKHAM@sakilacustomer.org	364	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	219	2	WILLIE	HOWELL	WILLIE.HOWELL@sakilacustomer.org	223	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	303	2	WILLIAM	SATTERFIELD	WILLIAM.SATTERFIELD@sakilacustomer.org	308	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	578	2	WILLARD	LUMPKIN	WILLARD.LUMPKIN@sakilacustomer.org	584	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	469	2	WESLEY	BULL	WESLEY.BULL@sakilacustomer.org	474	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	115	1	WENDY	HARRISON	WENDY.HARRISON@sakilacustomer.org	119	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	370	2	WAYNE	TRUONG	WAYNE.TRUONG@sakilacustomer.org	375	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	462	2	WARREN	SHERROD	WARREN.SHERROD@sakilacustomer.org	467	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	87	1	WANDA	PATTERSON	WANDA.PATTERSON@sakilacustomer.org	91	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- **오름차순 또는 내림차순으로 정렬하기**

- **ASC와 DESC를 조합하여 데이터 정렬하기**

- 오름차순, 내림차순을 각 열에 적용한 후 조합하여 정렬할 수도 있음
 - 각 열 이름 뒤에 ASC, DESC를 붙인 후 쉼표로 연결하면 됨
 - 다음 쿼리는 store_id 열은 내림차순, first_name 열은 오름차순으로 정렬함
 - 입력순으로 정렬 우선순위가 정해지므로 store_id 열을 먼저 정렬한 후, 같은 데이터가 있는 경우 first_name 열을 정렬함

- 오름차순 또는 내림차순으로 정렬하기
- ASC와 DESC를 조합하여 데이터 정렬하기

Do it! ASC와 DESC를 조합하여 데이터 정렬

```
SELECT * FROM customer ORDER BY store_id DESC, first_name ASC;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	375	2	AARON	SELBY	AARON.SELBY@sakilacustomer.org	380	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	525	2	ADRIAN	CLARY	ADRIAN.CLARY@sakilacustomer.org	531	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	217	2	AGNES	BISHOP	AGNES.BISHOP@sakilacustomer.org	221	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	568	2	ALBERTO	HENNING	ALBERTO.HENNING@sakilacustomer.org	574	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	454	2	ALEX	GRESHAM	ALEX.GRESHAM@sakilacustomer.org	459	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	439	2	ALEXANDER	FENINELL	ALEXANDER.FENINELL@sakilacustomer.org	444	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	423	2	ALFRED	CASILLAS	ALFRED.CASILLAS@sakilacustomer.org	428	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	567	2	ALFREDO	MCADAMS	ALFREDO.MCADAMS@sakilacustomer.org	573	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	412	2	ALLEN	BUTTERFIELD	ALLEN.BUTTERFIELD@sakilacustomer.org	417	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	228	2	ALLISON	STANLEY	ALLISON.STANLEY@sakilacustomer.org	232	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	467	2	ALVIN	DELOACH	ALVIN.DELOACH@sakilacustomer.org	472	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	40	2	AMANDA	CARTER	AMANDA.CARTER@sakilacustomer.org	44	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	181	2	ANA	BRADLEY	ANA.BRADLEY@sakilacustomer.org	185	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ LIMIT으로 상위 데이터 조회하기

- 특정 조건에 해당하는 데이터 중에서 상위 N개의 데이터만 보고 싶은 경우 SELECT 문에 LIMIT을 조합하면 됨
- 예를 들면 **SELECT ~ FROM ORDER BY ~ LIMIT 10**과 같은 방식으로 LIMIT 다음에 조회하려는 행의 개수를 입력하면 됨
- LIMIT의 경우 상위 N개의 데이터를 반환하므로 정렬 우선순위가 매우 중요함
- 다음은 store_id는 내림차순으로, first_name은 오름차순으로 정렬한 다음, 상위 10개의 데이터를 조회하는 쿼리임

04-3

ORDER BY 문으로 데이터 정렬하기

- LIMIT으로 상위 데이터 조회하기

Do it! LIMIT으로 상위 10개의 데이터 조회

```
SELECT * FROM customer ORDER BY store_id DESC, first_name ASC
```

LIMIT 10;

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	375	2	AARON	SELBY	AARON.SELBY@sakilacustomer.org	380	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	525	2	ADRIAN	CLARY	ADRIAN.CLARY@sakilacustomer.org	531	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	217	2	AGNES	BISHOP	AGNES.BISHOP@sakilacustomer.org	221	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	568	2	ALBERTO	HENNING	ALBERTO.HENNING@sakilacustomer.org	574	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	454	2	ALEX	GRESHAM	ALEX.GRESHAM@sakilacustomer.org	459	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	439	2	ALEXANDER	FENNELL	ALEXANDER.FENNELL@sakilacustomer.org	444	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	423	2	ALFRED	CASILLAS	ALFRED.CASILLAS@sakilacustomer.org	428	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	567	2	ALFREDO	MCADAMS	ALFREDO.MCADAMS@sakilacustomer.org	573	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	412	2	ALLEN	BUTTERFIELD	ALLEN.BUTTERFIELD@sakilacustomer.org	417	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	228	2	ALLISON	STANLEY	ALLISON.STANLEY@sakilacustomer.org	232	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- 상위 N개의 데이터를 조회할 때는 특히 ORDER BY 문을 사용하는 것이 좋음.
만약 정렬하지 않으면 어떤 값을 기준으로 정렬해
상위 N개의 데이터를 조회했는지 알 수 없기 때문

- LIMIT으로 상위 데이터 조회하기

- 범위를 지정해 데이터 조회하기

- LIMIT 문에 매개변수 1개만 입력했을 때에는 지정한 열에서 상위 N개만 조회함
 - 그런데 LIMIT과 함께 매개변수 2개를 입력하면 범위를 지정해 데이터를 조회할 수 있음
 - 예를 들어 N1, N2라는 매개변수를 입력했다면
상위 N1 다음 행부터 N2개의 행을 조회하게 됨

- LIMIT으로 상위 데이터 조회하기

- 범위를 지정해 데이터 조회하기

- 다음 쿼리는 customer_id 열을 기준으로 오름차순한 뒤, 101번째 행부터 10개 행을 조회함

Do it! LIMIT으로 101번째부터 10개의 데이터 조회

```
SELECT * FROM customer ORDER BY customer_id ASC LIMIT 100, 10;
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	101	1	PEGGY	MYERS	PEGGY.MYERS@sakilacustomer.org	105	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	102	1	CRYSTAL	FORD	CRYSTAL.FORD@sakilacustomer.org	106	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	103	1	GLADYS	HAMILTON	GLADYS.HAMILTON@sakilacustomer.org	107	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	104	1	RITA	GRAHAM	RITA.GRAHAM@sakilacustomer.org	108	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	105	1	DAWN	SULLIVAN	DAWN.SULLIVAN@sakilacustomer.org	109	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	106	1	CONNIE	WALLACE	CONNIE.WALLACE@sakilacustomer.org	110	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	107	1	FLORENCE	WOODS	FLORENCE.WOODS@sakilacustomer.org	111	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	108	1	TRACY	COLE	TRACY.COLE@sakilacustomer.org	112	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	109	2	EDNA	WEST	EDNA.WEST@sakilacustomer.org	113	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	110	2	TIFFANY	JORDAN	TIFFANY.JORDAN@sakilacustomer.org	114	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- LIMIT으로 상위 데이터 조회하기

- OFFSET으로 특정 구간의 데이터 조회하기

- ORDER BY로 데이터를 정렬한 다음,
상위나 하위가 아닌 특정 구간의 데이터를 조회해야 하는 경우도 있음
 - 이럴 때는 LIMIT ~ OFFSET을 사용함
 - OFFSET에 지정한 행 개수만큼 건너뛰고
LIMIT에 지정한 개수만큼의 상위 데이터를 출력함

- LIMIT으로 상위 데이터 조회하기

- OFFSET으로 특정 구간의 데이터 조회하기

- 다음은 customer_id 열을 기준으로 정렬한 다음,
데이터 100개를 건너뛰고 101번째 데이터부터 행 10개를 조회하는 쿼리임

Do it! 데이터 100 개를 건너뛰고 데이터 10개 조회

```
SELECT * FROM customer ORDER BY customer_id ASC LIMIT 10
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	101	1	PEGGY	MYERS	PEGGY.MYERS@sakilacustomer.org	105	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	102	1	CRYSTAL	FORD	CRYSTAL.FORD@sakilacustomer.org	106	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	103	1	GLADYS	HAMILTON	GLADYS.HAMILTON@sakilacustomer.org	107	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	104	1	RITA	GRAHAM	RITA.GRAHAM@sakilacustomer.org	108	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	105	1	DAWN	SULLIVAN	DAWN.SULLIVAN@sakilacustomer.org	109	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	106	1	CONNIE	WALLACE	CONNIE.WALLACE@sakilacustomer.org	110	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	107	1	FLORENCE	WOODS	FLORENCE.WOODS@sakilacustomer.org	111	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	108	1	TRACY	COLE	TRACY.COLE@sakilacustomer.org	112	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	109	2	EDNA	WEST	EDNA.WEST@sakilacustomer.org	113	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	110	2	TIFFANY	JORDAN	TIFFANY.JORDAN@sakilacustomer.org	114	1	2006-02-14 22:04:36	2006-02-15 04:57:20

- LIMIT으로 상위 데이터 조회하기
 - OFFSET으로 특정 구간의 데이터 조회하기
 - OFFSET은 반드시 LIMIT 문을 함께 사용해야 한다는 점에 주의해야 함
 - LIMIT는 상위 N개 행을 반환하기 때문에 정렬 기준을 명확히 하기 위해 ORDER BY를 반드시 함께 사용해야 함

- LIKE는 와일드카드로 지정한 패턴과 일치하는 문자열, 날짜, 시간 등을 조회함
- LIKE의 기본 형식

LIKE의 기본 형식

```
SELECT [열] FROM [테이블] WHERE [열] LIKE [조건값]
```

① ② ③

① [열]: 조건을 적용할 열 이름을 입력한다.

② LIKE: 조건값에 일치하는 데이터를 조회하는 구문이다.

③ [조건값]: 검색하고자 하는 데이터의 조건값을 입력한다.

▪ LIKE와 %로 특정 문자열을 포함하는 데이터 조회하기

- 특정 문자열을 포함하는 문자열을 조회할 때 %를 사용함
- %는 0개 이상의 문자열을 대치함
- % 위치에 따라 특정 문자열이 포함된 문자열을 조회할 수 있음

%의 사용 예

- A%: A로 시작하는 모든 문자열
- %A: A로 끝나는 모든 문자열
- %A%: A가 포함된 모든 문자열

- LIKE와 %로 특정 문자열을 포함하는 데이터 조회하기

1. 다음은 customer 테이블의 first_name 열에서 A로 시작하는 문자열 데이터를 조회하는 쿼리임.

실행 결과를 살펴보면
첫 번째 글자가 A로 시작하는 문자열이
조회된 것을 확인할 수 있음

▪ LIKE와 %로 특정 문자열을 포함하는 데이터 조회하기

1.

Do it! 첫 번째 글자가 A로 시작하는 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE 'A%';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	29	2	ANGELA	HERNANDEZ	ANGELA.HERNANDEZ@sakilacustomer.org	33	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	32	1	AMY	LOPEZ	AMY.LOPEZ@sakilacustomer.org	36	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	33	2	ANNA	HILL	ANNA.HILL@sakilacustomer.org	37	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	40	2	AMANDA	CARTER	AMANDA.CARTER@sakilacustomer.org	44	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	48	1	ANN	EVANS	ANN.EVANS@sakilacustomer.org	52	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	51	1	ALICE	STEWART	ALICE.STEWART@sakilacustomer.org	55	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	63	1	ASHLEY	RICHARDSON	ASHLEY.RICHARDSON@sakilacustomer.org	67	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	81	1	ANDREA	HENDERSON	ANDREA.HENDERSON@sakilacustomer.org	85	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	85	2	ANNE	POWELL	ANNE.POWELL@sakilacustomer.org	89	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	97	2	ANNIE	RUSSELL	ANNIE.RUSSELL@sakilacustomer.org	101	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	136	2	ANITA	MORALES	ANITA.MORALES@sakilacustomer.org	140	1	2006-02-14 22:04:36	2006-02-15 04:57:20

와일드카드로 문자열 조회하기

2. 다음은 first_name 열에서 AA로 시작하는 모든 문자열 데이터를 조회하는 조회하는 쿼리임

```
SELECT * FROM customer WHERE first name LIKE 'AA%';
```

[illegible]

▪ LIKE와 %로 특정 문자열을 포함하는 데이터 조회하기

3. 다음은 first_name 열에서 A로 끝나는 모든 문자열 데이터를 조회하는 조회하는 쿼리임

Do it! A로 끝나는 모든 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE '%A';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	6	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	11	2	LISA	ANDERSON	LISA.ANDERSON@sakilacustomer.org	15	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	16	2	SANDRA	MARTIN	SANDRA.MARTIN@sakilacustomer.org	20	0	2006-02-14 22:04:36	2006-02-15 04:57:20
	17	1	DONNA	THOMPSON	DONNA.THOMPSON@sakilacustomer.org	21	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	22	1	LAURA	RODRIGUEZ	LAURA.RODRIGUEZ@sakilacustomer.org	26	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	26	2	JESSICA	HALL	JESSICA.HALL@sakilacustomer.org	30	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	28	1	CYNTHIA	YOUNG	CYNTHIA.YOUNG@sakilacustomer.org	32	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	29	2	ANGELA	HERNANDEZ	ANGELA.HERNANDEZ@sakilacustomer.org	33	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	30	1	MELISSA	KING	MELISSA.KING@sakilacustomer.org	34	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ LIKE와 %로 특정 문자열을 포함하는 데이터 조회하기

4. 이번에는 같은 열에서 RA로 끝나는 모든 문자열 데이터를 조회하는 조회한 쿼리임

Do it! RA로 끝나는 모든 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE '%RA';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	16	2	SANDRA	MARTIN	SANDRA.MARTIN@sakilacustomer.org	20	0	2006-02-14 22:04:36	2006-02-15 04:57:20
	22	1	LAURA	RODRIGUEZ	LAURA.RODRIGUEZ@sakilacustomer.org	26	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	39	1	DEBRA	NELSON	DEBRA.NELSON@sakilacustomer.org	43	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	84	2	SARA	PERRY	SARA.PERRY@sakilacustomer.org	88	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	144	1	CLARA	SHAW	CLARA.SHAW@sakilacustomer.org	148	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	203	1	TARA	RYAN	TARA.RYAN@sakilacustomer.org	207	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	218	1	VERA	MCCOY	VERA.MCCOY@sakilacustomer.org	222	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	229	1	TAMARA	NGUYEN	TAMARA.NGUYEN@sakilacustomer.org	233	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	249	2	DORA	MEDINA	DORA.MEDINA@sakilacustomer.org	253	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	266	2	NORA	HERRERA	NORA.HERRERA@sakilacustomer.org	271	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	269	1	CASSANDRA	WALTERS	CASSANDRA.WALTERS@sakilacustomer.org	274	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ LIKE와 %로 특정 문자열을 포함하는 데이터 조회하기

5. A를 포함한 모든 first_name 열의 데이터를 조회하는 쿼리도 입력해 보자

Do it! A를 포함한 모든 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE '%A%';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	5	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	6	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	11	2	LISA	ANDERSON	LISA.ANDERSON@sakilacustomer.org	15	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	12	1	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org	16	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	13	2	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org	17	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	16	2	SANDRA	MARTIN	SANDRA.MARTIN@sakilacustomer.org	20	0	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ LIKE와 %로 특정 문자열을 포함하는 데이터 조회하기

6. 특정 문자열을 제외해 데이터를 조회하고 싶다면 NOT을 조합해 사용하면 됨.
다음 예를 통해 A로 시작하는 문자열을 제외한 first_name 열의 데이터를 조회함

Do it! 첫 번째 글자가 A로 시작하지 않는 데이터만 조회

```
SELECT * FROM customer WHERE first_name NOT LIKE 'A%';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	5	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	2	1	PATRICIA	JOHNSON	PATRICIA.JOHNSON@sakilacustomer.org	6	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	5	1	ELIZABETH	BROWN	ELIZABETH.BROWN@sakilacustomer.org	9	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	6	2	JENNIFER	DAVIS	JENNIFER.DAVIS@sakilacustomer.org	10	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	10	1	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org	14	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ ESCAPE로 특수 문자를 포함한 데이터 조회하기

- LIKE를 사용해 문자열에 특수 문자 %가 포함된 데이터를 검색하려면 어떻게 해야 할까 ?
- %는 예약어이므로 %%와 같이 입력하는 방법으로는 검색할 수 없음
- 만약 %를 포함한 데이터를 검색하고 싶다면 ESCAPE를 사용함

■ ESCAPE로 특수 문자를 포함한 데이터 조회하기

1. 먼저 %나 _와 같은 특수 기호를 포함한 임의의 테이블을 생성한 뒤, 실습을 진행해 보자.

이때, 공통 테이블 표현식을 활용해 이러한 테이블을 생성할 수 있음.
다음과 같이 쿼리를 입력해 데이터를 조회해 보자

Do it! 특수 문자를 포함한 임의의 테이블 생성

```
WITH CTE (col_1) AS (  
  SELECT 'A%BC' UNION ALL  
  SELECT 'A_BC' UNION ALL  
  SELECT 'ABC'  
)  
SELECT * FROM CTE;
```

실행 결과

	col_1
▶	A%BC
	A_BC
	ABC

▪ ESCAPE로 특수 문자를 포함한 데이터 조회하기

2. 이 테이블에서 특수 문자 %가 들어간 데이터를 조회해 보자.
다음과 같이 쿼리를 작성해 실행하면 어떻게 될까?

Do it! 특수 문자 %를 포함한 데이터 조회

```
WITH CTE (col_1) AS (  
  SELECT 'A%BC' UNION ALL  
  SELECT 'A_BC' UNION ALL  
  SELECT 'ABC'  
)  
  
SELECT * FROM CTE WHERE col_1 LIKE  
'%';
```

실행 결과

	col_1
▶	A%BC
	A_BC
	ABC

■ ESCAPE로 특수 문자를 포함한 데이터 조회하기

2. 그런데 이 결과는 우리가 원하는 결과가 아님.
이런 결과가 나온 이유는 % 기호는 검색할 수 있는 값이 아니라
0개 이상의 문자를 의미하는 예약어이기 때문.

% 기호가 포함된 데이터인 A%BC를 조회하려면 어떻게 해야 할까?
ESCAPE를 이용하면 됨

Do it! ESCAPE로 특수 문자 %를 포함한 데이터 조회

```
WITH CTE (col_1) AS (  
  SELECT 'A%BC' UNION ALL  
  SELECT 'A_BC' UNION ALL  
  SELECT 'ABC'  
)
```

```
SELECT * FROM CTE WHERE col_1 LIKE '%#%#%'  
ESCAPE '#';
```

실행 결과

	col_1
▶	A%BC

- **ESCAPE로 특수 문자를 포함한 데이터 조회하기**

2. 쿼리를 실행할 때 ESCAPE 문이 #을 제거해
쿼리 명령 단계에서는 %#%%이 호출되고

실제 실행할 때는 %%%로 해석되어
%를 포함하는 앞뒤 어떠한 문자가 와도 상관없는 데이터가 검색되는 것임

■ ESCAPE로 특수 문자를 포함한 데이터 조회하기

3. ESCAPE에 사용할 특수문자는 # 외에 &, !, / 등 다른 것도 쓸 수 있음.

다만 해당 문자가 데이터에 쓰이지 않는 것이어야
의도하지 않은 데이터 오류를 방지할 수 있음

Do it! ESCAPE와 !로 특수 문자 %를 포함한 데이터 조회

```
WITH CTE (col_1) AS (  
  SELECT 'A%BC' UNION ALL  
  SELECT 'A_BC' UNION ALL  
  SELECT 'ABC'  
)  
SELECT * FROM CTE WHERE col_1 LIKE '%!%%' ESCAPE '!';
```

실행 결과

	col_1
▶	A%BC

▪ ESCAPE로 특수 문자를 포함한 데이터 조회하기

3. 이와 같이 #이 아닌 !로도 원하는 데이터를 조회할 수 있음.

이 과정을 쪼개서 살펴보면 좀 더 이해가 쉬움

명령어가 전달될 때

```
SELECT * FROM CTE WHERE col_1 LIKE '%!%%' ESCAPE '!';
```

데이터베이스 엔진이 SQL 명령을 수행할 때

```
SELECT * FROM CTE WHERE col_1 LIKE '%%';
```

▪ LIKE와 _로 길이가 정해진 데이터 조회하기

- %는 특정 문자열을 포함하는 데이터를 검색함
- 특정 문자열을 포함하면서 문자열의 길이도 정해 데이터를 검색하려면 _를 사용함
- %만 사용하면 검색된 데이터의 양이 많아 원하는 데이터를 빠르게 찾지 못할 수 있음

_의 사용 예

- A_: A로 시작하면서 뒤의 글자는 무엇이든 상관없으며 전체 글자 수는 2개인 문자열
- _A: A로 끝나면서 앞의 문자는 무엇이든 상관없으며 전체 글자 수는 2개인 문자열
- _A_: 세 글자로 된 문자열 중 가운데 글자만 A이며 앞뒤로는 무엇이든 상관없는 문자열

와일드카드로 문자열 조회하기

1. A로 시작하면서 뒤의 글자가 하나 있는 문자열을 customer 테이블의 first_name 열에서 조회해 보자

```
SELECT * FROM customer WHERE first name LIKE 'A ';
```

[illegible]

▪ LIKE와 _로 길이가 정해진 데이터 조회하기

2. 이번에는 A로 시작하면서 문자열 길이가 3인 데이터를 first_name 열에서 조회함.
조건에 맞는 행이 있어서 다음과 같이 3건이 출력됨

Do it! A로 시작하면서 문자열 길이가 3인 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE 'A__';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	32	1	AMY	LOPEZ	AMY.LOPEZ@sakilacustomer.org	36	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	48	1	ANN	EVANS	ANN.EVANS@sakilacustomer.org	52	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	181	2	ANA	BRADLEY	ANA.BRADLEY@sakilacustomer.org	185	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ LIKE와 _로 길이가 정해진 데이터 조회하기

3. 반대로 A로 끝나면서 문자열 길이가 3인 데이터를 first_name 열에서 조회해 보자

Do it! A로 끝나면서 문자열 길이가 3인 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE '__A';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	140	1	EVA	RAMOS	EVA.RAMOS@sakilacustomer.org	144	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	181	2	ANA	BRADLEY	ANA.BRADLEY@sakilacustomer.org	185	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	183	2	IDA	ANDREWS	IDA.ANDREWS@sakilacustomer.org	187	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ LIKE와 _로 길이가 정해진 데이터 조회하기

4. A로 시작하고 A로 끝나면서 문자열 길이가 4인 데이터를 조회해 보자

Do it! A로 시작하고 A로 끝나면서 문자열 길이가 4인 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE 'A__A';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	33	2	ANNA	HILL	ANNA.HILL@sakilacustomer.org	37	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	196	1	ALMA	AUSTIN	ALMA.AUSTIN@sakilacustomer.org	200	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ LIKE와 _로 길이가 정해진 데이터 조회하기

5. 다음과 같이 5글자인 이름을 찾으려면 _ 문자를 5번 입력해 데이터를 조회할 수도 있습니다

Do it! 문자열 길이가 5인 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE '_____';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	3	1	LINDA	WILLIAMS	LINDA.WILLIAMS@sakilacustomer.org	7	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	12	1	NANCY	THOMAS	NANCY.THOMAS@sakilacustomer.org	16	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	13	2	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org	17	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	14	2	BETTY	WHITE	BETTY.WHITE@sakilacustomer.org	18	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	15	1	HELEN	HARRIS	HELEN.HARRIS@sakilacustomer.org	19	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	17	1	DONNA	THOMPSON	DONNA.THOMPSON@sakilacustomer.org	21	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	18	2	CAROL	GARCIA	CAROL.GARCIA@sakilacustomer.org	22	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	22	1	LAURA	RODRIGUEZ	LAURA.RODRIGUEZ@sakilacustomer.org	26	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	23	2	SARAH	LEWIS	SARAH.LEWIS@sakilacustomer.org	27	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	30	1	DEBRA	MELSON	DEBRA.MELSON@sakilacustomer.org	43	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ _과 %로 문자열 조회하기

- 특정 문자열을 포함하고 길이가 정해진 데이터를 조회하려면 _과 %를 조합해 사용하면 됨
1. 다음 A와 R 사이에 한 글자를 포함하여 시작하면서 이후 문자열은 어떤 문자열이라도 상관없는 데이터를 조회해 보자

Do it! A_R로 시작하는 문자열 조회

```
SELECT * FROM customer WHERE first_name LIKE 'A_R%';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	142	1	APRIL	BURNS	APRIL.BURNS@sakilacustomer.org	146	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	375	2	AARON	SELBY	AARON.SELBY@sakilacustomer.org	380	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	525	2	ADRIAN	CLARY	ADRIAN.CLARY@sakilacustomer.org	531	1	2006-02-14 22:04:37	2006-02-15 04:57:20

▪ _과 %로 문자열 조회하기

2. 다음은 R 앞에 두 글자를 포함하여 시작하면서 이후 문자열은 어떤 문자열이라도 상관없는 데이터를 조회함

Do it! __R로 시작하는 문자열 조회

```
SELECT * FROM customer WHERE first_name LIKE '__R%';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	1	1	MARY	SMITH	MARY.SMITH@sakilacustomer.org	5	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	4	2	BARBARA	JONES	BARBARA.JONES@sakilacustomer.org	8	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	7	1	MARIA	MILLER	MARIA.MILLER@sakilacustomer.org	11	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	9	2	MARGARET	MOORE	MARGARET.MOORE@sakilacustomer.org	13	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	10	1	DOROTHY	TAYLOR	DOROTHY.TAYLOR@sakilacustomer.org	14	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	13	2	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org	17	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	18	2	CAROL	GARCIA	CAROL.GARCIA@sakilacustomer.org	22	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	23	2	SARAH	LEWIS	SARAH.LEWIS@sakilacustomer.org	27	1	2006-02-14 22:04:36	2006-02-15 04:57:20

와일드카드로 문자열 조회하기

3. 이번에는 A로 시작하고, 마지막은 R과 함께
마지막 한 글자가 더 있는 문자열 데이터를 조회해 보자

```
SELECT * FROM customer WHERE first name LIKE 'A%R ';
```

[illegible]

- **REGEXP로 더 다양하게 데이터 조회하기**

- MySQL에서는 정규 표현식(regular expression)을 의미하는 REGEXP으로 더 다양한 방법으로 문자열을 검색할 수 있음
- 정규 표현식은 특정한 패턴의 문자열을 표현하기 위해 사용함

▪ REGEXP로 더 다양하게 데이터 조회하기

- MySQL에서 지원하는 정규 표현식

표현식	설명
.	줄 바꿈 문자(Wn)를 제외한 임의의 한 문자를 의미한다.
*	해당 문자 패턴이 0번 이상 반복한다.
+	해당 문자 패턴이 1번 이상 반복한다.
^	문자열의 처음을 의미한다.
\$	문자열의 끝을 의미한다.
	OR를 의미한다.
[...]	대괄호([]) 안에 있는 어떠한 문자를 의미한다.
[^...]	대괄호([]) 안에 있지 않은 어떠한 문자를 의미한다.
{n}	반복되는 횟수를 지정한다.
{m,n}	반복되는 횟수의 최솟값과 최댓값을 지정한다.

REGEXP로 더 다양하게 데이터 조회하기

- 정규 표현식을 사용해 customer 테이블의 first_name 열에서 K로 시작하거나 N으로 끝나는 데이터를 조회해 보자

Do it! ^, |, \$를 사용해 데이터 조회

```
SELECT * FROM customer WHERE first_name REGEXP '^K|N$';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	13	2	KAREN	JACKSON	KAREN.JACKSON@sakilacustomer.org	17	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	15	1	HELEN	HARRIS	HELEN.HARRIS@sakilacustomer.org	19	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	20	2	SHARON	ROBINSON	SHARON.ROBINSON@sakilacustomer.org	24	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	24	2	KIMBERLY	LEE	KIMBERLY.LEE@sakilacustomer.org	28	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	36	2	KATHLEEN	ADAMS	KATHLEEN.ADAMS@sakilacustomer.org	40	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	42	2	CAROLYN	PEREZ	CAROLYN.PEREZ@sakilacustomer.org	46	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	48	1	ANN	EVANS	ANN.EVANS@sakilacustomer.org	52	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	57	2	EVELYN	MORGAN	EVELYN.MORGAN@sakilacustomer.org	61	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	58	1	JEAN	BELL	JEAN.BELL@sakilacustomer.org	62	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	61	2	KATHERINE	RIVERA	KATHERINE.RIVERA@sakilacustomer.org	65	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	62	1	JOAN	COOPER	JOAN.COOPER@sakilacustomer.org	66	1	2006-02-14 22:04:36	2006-02-15 04:57:20

▪ REGEXP로 더 다양하게 데이터 조회하기

2. first_name 열에서 K와 함께 L과 N 사이의 글자를 포함한 데이터를 조회해 보자.

Do it! [...]를 사용해 데이터 조회

```
SELECT * FROM customer WHERE first_name REGEXP 'K[L-N]';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	514	2	FRANKLIN	TROUTMAN	FRANKLIN.TROUTMAN@sakilacustomer.org	520	1	2006-02-14 22:04:37	2006-02-15 04:57:20

결과를 살펴보면 first_name 열의 문자열 데이터에서 K 뒤에 L과 N사이의 글자가 포함되어 있는 데이터를 검색하여 결과를 보여줌

▪ REGEXP로 더 다양하게 데이터 조회하기

3. 반대로 K와 함께 L과 N 사이의 글자를 포함하지 않는 데이터를 조회해 보자.

Do it! [^...]를 사용해 데이터 조회

```
SELECT * FROM customer WHERE first_name REGEXP 'K[^L-N]';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
	357	1	KEITH	RICO	KEITH.RICO@sakilacustomer.org	362	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	403	1	MIKE	WAY	MIKE.WAY@sakilacustomer.org	408	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	424	2	KYLE	SPURLOCK	KYLE.SPURLOCK@sakilacustomer.org	429	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	435	2	RICKY	SHELBY	RICKY.SHELBY@sakilacustomer.org	440	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	526	2	KARL	SEAL	KARL.SEAL@sakilacustomer.org	532	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	546	1	KELLY	KNOTT	KELLY.KNOTT@sakilacustomer.org	552	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	547	1	KURT	EMMONS	KURT.EMMONS@sakilacustomer.org	553	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	563	2	KEN	PREWITT	KEN.PREWITT@sakilacustomer.org	569	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	586	1	KIRK	STCLAIR	KIRK.STCLAIR@sakilacustomer.org	592	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	591	1	KENT	ARSENAULT	KENT.ARSENAULT@sakilacustomer.org	597	1	2006-02-14 22:04:37	2006-02-15 04:57:20
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

결과를 살펴보면 first_name 열의 문자열에서 K 뒤에 L과 N사이의 글자가 포함되어 있지 않는 데이터를 검색하여 결과를 보여줌

와일드카드 더 활용해 보기

- 다음은 S로 시작하는 문자열 데이터 중에
A 뒤에 L과 N 사이의 글자가 있는 데이터를 조회함

Do it! %와 [...]을 사용해 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE 'S%' AND first_name
DECEYD 'A' N'.
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
	8	2	SUSAN	WILSON	SUSAN.WILSON@sakilacustomer.org	12	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	16	2	SANDRA	MARTIN	SANDRA.MARTIN@sakilacustomer.org	20	0	2006-02-14 22:04:36	2006-02-15 04:57:20
	41	1	STEPHANIE	MITCHELL	STEPHANIE.MITCHELL@sakilacustomer.org	45	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	123	2	SHANNON	FREEMAN	SHANNON.FREEMAN@sakilacustomer.org	127	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	153	2	SUZANNE	NICHOLS	SUZANNE.NICHOLS@sakilacustomer.org	157	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	167	2	SALLY	PIERCE	SALLY.PIERCE@sakilacustomer.org	171	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	177	2	SAMANTHA	DUNCAN	SAMANTHA.DUNCAN@sakilacustomer.org	181	1	2006-02-14 22:04:36	2006-02-15 04:57:20
	358	2	SAMUEL	MARLOW	SAMUEL.MARLOW@sakilacustomer.org	363	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	392	2	SEAN	DOUGLASS	SEAN.DOUGLASS@sakilacustomer.org	397	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	404	2	STANLEY	SCROGGINS	STANLEY.SCROGGINS@sakilacustomer.org	409	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	488	2	SHANE	MILLARD	SHANE.MILLARD@sakilacustomer.org	493	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	490	1	SAM	MCDUFFEE	SAM.MCDUFFEE@sakilacustomer.org	495	1	2006-02-14 22:04:37	2006-02-15 04:57:20

■ 와일드카드 더 활용해 보기

2. 좀 더 복잡한 조건으로 데이터를 조회해 보자.
다음은 first_name 열에서 총 7 글자이고, A 뒤에 L과 N 사이의 글자가 있고,
마지막 글자는 O인 문자열 데이터를 조회하는 쿼리임

Do it! 와일드카드 조합으로 데이터 조회

```
SELECT * FROM customer WHERE first_name LIKE '_____'
AND first_name REGEXP 'A[L-N]'
AND first_name REGEXP 'O$';
```

실행 결과

	customer_id	store_id	first_name	last_name	email	address_id	active	create_date	last_update
▶	398	1	ANTONIO	MEEK	ANTONIO.MEEK@sakilacustomer.org	403	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	556	2	ARMANDO	GRUBER	ARMANDO.GRUBER@sakilacustomer.org	562	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	567	2	ALFREDO	MCADAMS	ALFREDO.MCADAMS@sakilacustomer.org	573	1	2006-02-14 22:04:37	2006-02-15 04:57:20
	568	2	ALBERTO	HENNING	ALBERTO.HENNING@sakilacustomer.org	574	1	2006-02-14 22:04:37	2006-02-15 04:57:20

04-5

GROUP BY 문으로 데이터 묶기

- 조회한 데이터들을 그룹으로 묶어 정보를 확인해야 할 때가 있고, 데이터 그룹에 특정 조건을 필터링해 필요한 데이터를 조회해야 하는 경우도 있음
- 예를 들어 다음과 같이 어떤 회사의 직원 정보가 담긴 테이블이 있다고 하자
- 이를 소속별로 그룹화하고 싶다면 다음 표와 같이 묶을 수 있음

이름	직급	소속
김지은	대리	마케팅
이선우	대리	경영지원
박수연	과장	마케팅
안성민	차장	개발
정서경	부장	개발



소속
마케팅
경영지원
개발
개발

- 이렇게 데이터를 그룹화하고 싶을 때 바로 GROUP BY 문을 사용하면 됨
- 데이터 그룹을 필터링할 때는 HAVING 문을 사용함
- GROUP BY 문, HAVING 문은 다음과 같이 사용함

GROUP BY 문과 HAVING 문의 기본 형식

```
SELECT [열] FROM [테이블] WHERE [열] = [조건값] GROUP BY [열] HAVING [열] = [조건값]
```

① ② ③ ④ ⑤

- ① GROUP BY: 특정 열의 데이터를 그룹화하겠다는 구문이다.
- ②, ④ [열]: 그룹화된 데이터들이 있는 열 이름을 지정한다.
- ③ [HAVING]: WHERE와 비슷한 역할을 하며 그룹화된 데이터를 필터링할 수 있다.
- ⑤ [조건값]: 필터링을 위한 조건값을 입력한다.

- **GROUP BY 문으로 데이터 그룹화하기**
 - GROUP BY 문은 지정한 열에 있는 데이터를 그룹화함
 - film 테이블에 저장되어 있는 special_features 열의 데이터 종류가 무엇인지 궁금하다면 GROUP BY 문을 사용하면 됨

GROUP BY 문으로 데이터 그룹화하기

하나의 열을 기준으로 그룹화하기

- 데이터를 그룹화할 때는 반드시 그룹화할 기준 열을 지정해야 함
- 다음은 special_features, rating 열을 기준으로 각각 데이터를 그룹화한 것임
- 즉, film 테이블에서 special_features 열과 rating 열에 어떤 데이터 그룹이 있는지 확인하는 쿼리임

Do it! special_features 열의 데이터를 그룹화

```
SELECT special_features FROM film GROUP BY  
special_features;
```

실행 결과

special_features
Deleted Scenes,Behind the Scenes
Trailers,Deleted Scenes
Commentaries,Behind the Scenes
Deleted Scenes
Trailers
Commentaries,Deleted Scenes
Trailers,Deleted Scenes,Behind the Scenes
Trailers,Commentaries,Behind the Scenes
Trailers,Commentaries
Trailers,Behind the Scenes
Commentaries,Deleted Scenes,Behind the Scenes
Trailers,Commentaries,Deleted Scenes

- GROUP BY 문으로 데이터 그룹화하기

- 하나의 열을 기준으로 그룹화하기

Do it! rating 열의 데이터를 그룹화

```
SELECT rating FROM film GROUP BY rating;
```

실행 결과	
	rating
▶	PG
	G
	NC-17
	PG-13
	R

- special_features 열에서는 Deleted Scenes, Trailers 등으로 데이터가 그룹화되었고, rating 열에서는 PG, G 등으로 데이터가 그룹화된 것을 확인할 수 있음

- **GROUP BY 문으로 데이터 그룹화하기**
 - **2개 이상의 열을 기준으로 그룹화하기**
 - 2개 이상의 열을 기준으로 그룹화하는 방법도 알아보자
 - 열을 2개 이상 지정하여 그룹화할 때는
GROUP BY에 열을 순서대로 작성하면 그 순서대로 데이터를 그룹화함

GROUP BY 문으로 데이터 그룹화하기

2개 이상의 열을 기준으로 그룹화하기

1. special_features, rating 열 순서로 그룹화하면
special_features 열로 우선 그룹화한 다음 rating 열로 그룹화한 결과를 출력함

Do it! special_features, rating 열 순서로 데이터를 그룹화

```
SELECT special_features, rating FROM film GROUP BY  
special_features, rating;
```

실행 결과

special_features	rating
Deleted Scenes,Behind the Scenes	PG
Trailers,Deleted Scenes	G
Trailers,Deleted Scenes	NC-17
Commentaries,Behind the Scenes	G
Deleted Scenes	G
Deleted Scenes	PG
Trailers,Deleted Scenes	PG-13
Trailers	R
Commentaries,Deleted Scenes	PG
Trailers,Deleted Scenes,Behind the Scenes	NC-17
Trailers,Commentaries,Behind the Scenes	NC-17
Trailers,Commentaries	NC-17

- GROUP BY 문으로 데이터 그룹화하기

- 2개 이상의 열을 기준으로 그룹화하기

2. 반대로 rating, special_features 열 순서로 데이터를 그룹화해 보자.

Do it! rating, special_features 열 순서로 데이터를 그룹화

```
SELECT rating, special_features FROM film  
GROUP BY special_features, rating;
```

실행 결과

	rating	special_features
▶	PG	Deleted Scenes,Behind the Scenes
	G	Trailers,Deleted Scenes
	NC-17	Trailers,Deleted Scenes
	G	Commentaries,Behind the Scenes
	G	Deleted Scenes
	PG	Deleted Scenes
	PG-13	Trailers,Deleted Scenes
	R	Trailers
	PG	Commentaries,Deleted Scenes
	NC-17	Trailers,Deleted Scenes,Behind the Scenes
	NC-17	Trailers,Commentaries,Behind the Scenes
	NC-17	Trailers,Commentaries

열 순서만 변경되었을 뿐 쿼리 결과를 보면 모두 같은 개수의 데이터(행)를 출력함

- GROUP BY 문으로 데이터 그룹화하기

- COUNT로 그룹화한 열의 데이터 개수 세기

1. special_features 열을 기준으로 데이터를 그룹화한 다음 COUNT 함수로 각 데이터 그룹에 속한 데이터가 몇 개인지 조회하는 쿼리를 작성해 보자.

이때 COUNT 함수에는 AS cnt를 붙였는데
이는 계산한 결과를 출력할 때 열 이름을 cnt로 하겠다는 의미임

- GROUP BY 문으로 데이터 그룹화하기
 - COUNT로 그룹화한 열의 데이터 개수 세기

1.

Do it! COUNT 함수로 그룹에 속한 데이터 개수 세기

```
SELECT special_features, COUNT(*) AS cnt FROM film GROUP BY
```

실행 결과

special_features	cnt
Deleted Scenes,Behind the Scenes	71
Trailers,Deleted Scenes	66
Commentaries,Behind the Scenes	70
Deleted Scenes	61
Trailers	72
Commentaries,Deleted Scenes	65
Trailers,Deleted Scenes,Behind the Scenes	49
Trailers,Commentaries,Behind the Scenes	79
Trailers,Commentaries	72
Trailers,Behind the Scenes	72
Commentaries,Deleted Scenes,Behind the Scenes	66
Trailers,Commentaries,Deleted Scenes	64

- GROUP BY 문으로 데이터 그룹화하기

- COUNT로 그룹화한 열의 데이터 개수 세기

2. 다음 쿼리는 special_features, rating 열을 기준으로 그룹화한 다음 각 그룹의 데이터가 몇 개인지 셀 수 있음.

이때 쿼리 마지막 부분에 ORDER BY 문을 입력한 이유는
조회한 데이터의 결과를 정렬하여 개수를 확인하기 쉽게 하기 위함임.

여기서는 그룹화한 데이터의 개수에 따라 내림차순으로 정렬함

- GROUP BY 문으로 데이터 그룹화하기

- COUNT로 그룹화한 열의 데이터 개수 세기

2.

Do it! 두 열의 데이터 그룹에 속한 데이터 개수 세기

```
SELECT special_features, rating, COUNT(*) AS cnt FROM film  
GROUP BY special_features, rating ORDER BY special_features, rating, cnt
```

실행 결과

	special_features	rating	cnt
▶	Trailers	G	16
	Trailers	PG	10
	Trailers	PG-13	19
	Trailers	R	15
	Trailers	NC-17	12
	Commentaries	G	13
	Commentaries	PG	12
	Commentaries	PG-13	11
	Commentaries	R	14
	Commentaries	NC-17	12
	Trailers,Commentaries	G	10
	Trailers,Commentaries	PG	15

- GROUP BY 문으로 데이터 그룹화하기

- COUNT로 그룹화한 열의 데이터 개수 세기

3. 지금까지 SELECT 문으로 조회한 열 이름이 GROUP BY 문에서 그대로 사용되었음을 파악했음.

데이터를 그룹화할 때는 기준이 되는 열이 필요하므로 반드시 SELECT 문에 사용한 열을 반드시 GROUP BY 문에도 사용해야 함.

만약 SELECT 문에 사용한 열을 GROUP BY 문에 사용하지 않으면 오류가 발생할 것임

SELECT 문과 GROUP BY 문의 열 이름을 달리할 경우

```
SELECT special_features, rating, COUNT(*) AS cnt FROM film GROUP BY rating;
```


- GROUP BY 문으로 데이터 그룹화하기

- COUNT로 그룹화한 열의 데이터 개수 세기

3. 오류를 살펴보면 다음과 같은 메시지가 출력됨.

```
Error Code: 1055. Expression #1 of SELECT list is not in GROUP BY clause and contains nonaggregated column 'sakila.film.special_features' which is not functionally dependent on columns in GROUP BY clause; this is incompatible with sql_mode=only_full_group_by
```

오류 내용은 SELECT 문에 사용된 열이
GROUP BY에 명시되지 않아 열을 그룹화할 수 없다는 뜻임

- **HAVING 문으로 그룹화한 데이터 필터링하기**

- 그룹화한 데이터에서 데이터를 필터링하려면 HAVING 문을 사용해야 함
- HAVING 문은 WHERE 문과 비슷하지만
WHERE 문은 테이블에 있는 열에 대해 적용하는 것이라면

HAVING 문은 SELECT 문으로 조회한 열이나 GROUP BY 문에 그룹화한 열에만 필터링을 적용할 수 있음

▪ HAVING 문으로 그룹화한 데이터 필터링하기

1. 다음은 special_features, rating 열을 기준으로 그룹화한 뒤, rating 열에서 G인 데이터만 필터링해 조회해 보자

Do it! rating 열에서 G인 데이터만 필터링

```
SELECT special_features, rating, FROM film  
GROUP BY special_features, rating  
HAVING rating = 'G';
```

실행 결과

special_features	rating
Trailers,Deleted Scenes	G
Commentaries,Behind the Scenes	G
Deleted Scenes	G
Trailers	G
Commentaries,Deleted Scenes	G
Behind the Scenes	G
Trailers,Commentaries,Deleted Scenes,Behind the Scenes	G
Trailers,Commentaries	G
Deleted Scenes,Behind the Scenes	G
Trailers,Behind the Scenes	G
Trailers.Commentaries.Behind the Scenes	G

▪ HAVING 문으로 그룹화한 데이터 필터링하기

2. COUNT 함수도 적용해 보자.
다음은 special_features 열을 기준으로 그룹화한 다음
각 그룹에서 데이터의 개수가 70보다 큰 것만 조회하는 쿼리임

Do it! special_features 열에서 데이터 개수가 70보다 큰 것만 필터링

```
SELECT special_features, COUNT(*) AS cnt FROM film  
GROUP BY special_features  
HAVING cnt > 70;
```

실행 결과

special_features	cnt
Deleted Scenes,Behind the Scenes	71
Trailers	72
Trailers,Commentaries,Behind the Scenes	79
Trailers,Commentaries	72
Trailers,Behind the Scenes	72

▪ HAVING 문으로 그룹화한 데이터 필터링하기

3. 만약 그룹화하지 않은 열에 HAVING 문을 사용하면 오류가 발생할 것임.

그룹화하지 않은 열을 필터링할 경우

```
SELECT special_features, COUNT(*) AS cnt FROM film  
GROUP BY special_features  
HAVING rating = 'G';
```

오류를 살펴보면 다음과 같은 메시지가 출력됨.

```
Error Code: 1054. Unknown column 'rating' in 'having clause'
```

오류 내용은 SELECT 문에 rating 열이 없어
HAVING으로 해당 열을 필터링할 수 없다는 내용임

▪ HAVING 문으로 그룹화한 데이터 필터링하기

3. 오류가 난 쿼리를 수정해 보자.

HAVING 문에 사용하는 열을 SELECT 문과 GROUP BY 문에 추가하면 됨

Do it! 오류 없이 필터링할 경우

```
SELECT special_features, rating, COUNT(*) AS cnt
FROM film
GROUP BY special_features, rating
HAVING rating = 'R' AND cnt > 8;
```

실행 결과

special_features	rating	cnt
Trailers	R	15
Trailers,Behind the Scenes	R	21
Commentaries,Deleted Scenes,Behind the Scenes	R	12
Trailers,Deleted Scenes	R	11
Trailers,Deleted Scenes,Behind the Scenes	R	9
Trailers,Commentaries	R	16
Trailers,Commentaries,Behind the Scenes	R	11
Trailers,Commentaries,Deleted Scenes	R	11
Deleted Scenes	R	12
Deleted Scenes,Behind the Scenes	R	17
Commentaries	R	14

- **DISTINCT 문으로 중복된 데이터 제거하기**

- GROUP BY 문을 사용하지 않고
같은 열에 있는 중복 데이터를 제거하고 싶다면
DISTINCT 문을 사용하면 됨

- DISTINCT 문의 기본 형식

DISTINCT 문의 기본 형식

```
SELECT DISTINCT [열] FROM [테이블]
```

- DISTINCT 문은 지정한 열의 중복 데이터를 제거함

▪ DISTINCT 문으로 중복된 데이터 제거하기

1. 다음은 DISTINCT 문으로 film 테이블의 special_features, rating 열에서 중복된 데이터를 제거하는 쿼리임.
결과를 보면 앞서 GROUP BY 문으로 얻은 결과와 같은 것을 확인할 수 있음

Do it! 두 열의 데이터 중복 제거

```
SELECT DISTINCT special_features, rating FROM  
film;
```

실행 결과

special_features	rating
Deleted Scenes,Behind the Scenes	PG
Trailers,Deleted Scenes	G
Trailers,Deleted Scenes	NC-17
Commentaries,Behind the Scenes	G
Deleted Scenes	G
Deleted Scenes	PG
Trailers,Deleted Scenes	PG-13
Trailers	R
Commentaries,Deleted Scenes	PG
Trailers,Deleted Scenes,Behind the Scenes	NC-17
Trailers,Commentaries,Behind the Scenes	NC-17

▪ DISTINCT 문으로 중복된 데이터 제거하기

1. 비교를 위해 GROUP BY 문의 결과를 살펴보자

Do it! GROUP BY 문으로 두 열을 그룹화한 경우

```
SELECT special_features, rating FROM film  
GROUP BY special_features, rating;
```

실행 결과

	special_features	rating
▶	Deleted Scenes,Behind the Scenes	PG
	Trailers,Deleted Scenes	G
	Trailers,Deleted Scenes	NC-17
	Commentaries,Behind the Scenes	G
	Deleted Scenes	G
	Deleted Scenes	PG
	Trailers,Deleted Scenes	PG-13
	Trailers	R
	Commentaries,Deleted Scenes	PG
	Trailers,Deleted Scenes,Behind the Scenes	NC-17
	Trailers,Commentaries,Behind the Scenes	NC-17

▪ DISTINCT 문으로 중복된 데이터 제거하기

2. GROUP BY 문을 사용했을 때와 결과가 같다고 해서 GROUP BY 문과 DISTINCT 문이 같은 역할을 한다고 착각하면 안 됨.

DISTINCT 문은 중복을 제거할 뿐이지
그룹화를 하거나 이를 이용해 계산 작업을 할 수는 없음.

만약 계산이 필요하다면 반드시 GROUP BY 문을 사용해야 함

DISTINCT 문에 COUNT 함수를 사용할 경우

```
SELECT DISTINCT special_features, rating, COUNT(*) AS cnt FROM  
film;
```

▪ DISTINCT 문으로 중복된 데이터 제거하기

2. 오류를 살펴보면 다음과 같은 메시지가 출력됨.

```
Error Code: 1140. In aggregated query without GROUP BY, expression #1 of  
SELECT list contains nonaggregated column 'sakila.film.special_features'; this  
is incompatible with sql_mode=only_full_group_by
```

오류 내용을 살펴보면 GROUP BY가 사용되지 않아 발생한 오류임을 알 수 있음.
COUNT 함수를 사용하여 그룹별로 집계를 하기 때문에 반드시 GROUP BY가 필요함

- 테이블 생성 시 사용할 수 있는 다양한 옵션을 살펴봄
- 이번 실습은 doitsql 데이터베이스에서 진행하므로 이 데이터베이스가 없다면 다음 쿼리문을 입력하여 데이터베이스를 먼저 생성한 뒤 시작하자

```
CREATE DATABASE IF NOT EXISTS doitsql;
```

■ AUTO_INCREMENT로 데이터 자동 입력하기

- 테이블을 생성할 때 어떤 열에 AUTO_INCREMENT를 적용하면 해당 열은 1씩 또는 사용자가 정의한 값만큼 증가하며 데이터가 자동으로 입력됨.

이때, AUTO_INCREMENT가 적용된 열을 생성할 때에는 반드시 해당 열을 기본키로 설정해야 함

- **AUTO_INCREMENT로 데이터 자동 입력하기**

- **AUTO_INCREMENT로 데이터 입력하기**

1. 다음 쿼리는 테이블을 생성할 때 AUTO_INCREMENT를 적용하여 데이터를 입력함.

테이블을 생성한 뒤, 조회해 보면 AUTO_INCREMENT가 적용된 열에 자동으로 1씩 증가한 데이터가 입력된 것을 확인할 수 있음

■ AUTO_INCREMENT로 데이터 자동 입력하기

■ AUTO_INCREMENT로 데이터 입력하기

1.

Do it! 첫 번째 열에 AUTO_INCREMENT 적용

```
USE doitsql;
```

```
CREATE TABLE doit_increment (  
col_1 INT AUTO_INCREMENT PRIMARY KEY,  
col_2 VARCHAR(50),  
col_3 INT  
);
```

```
INSERT INTO doit_increment (col_2, col_3) VALUES ('1 자동 입력', 1);  
INSERT INTO doit_increment (col_2, col_3) VALUES ('2 자동 입력', 2);
```

```
SELECT * FROM doit_increment;
```

실행 결과

	col_1	col_2	col_3
▶	1	1 자동 입력	1
	2	2 자동 입력	2

- **AUTO_INCREMENT로 데이터 자동 입력하기**

- **AUTO_INCREMENT로 데이터 입력하기**

1. 그런데 쿼리를 살펴보면 AUTO_INCREMENT가 설정된 열에는 값을 입력하지 않았음.
AUTO_INCREMENT에 의해 자동으로 값이 증가돼 입력되도록 정의되어 있기 때문임

▪ AUTO_INCREMENT로 데이터 자동 입력하기

▪ AUTO_INCREMENT로 데이터 입력하기

2. 만약 AUTO_INCREMENT가 정의된 열에 값을 입력하려고 하면 어떻게 되는지 알아보자

Do it! 자동 입력되는 값과 동일한 값을 입력한 경우

```
INSERT INTO doit_increment (col_1, col_2, col_3) VALUES (3, '3 자동  
입력', 3);
```

```
SELECT * FROM doit_increment;
```

자동 입력되는 값과 동일한 값을 입력한 경우
문제없이 쿼리가 실행되고 이와 같은 결과가 출력됨

실행 결과

	col_1	col_2	col_3
▶	1	1 자동 입력	1
	2	2 자동 입력	2
	3	3 자동 입력	3

■ AUTO_INCREMENT로 데이터 자동 입력하기

■ AUTO_INCREMENT로 데이터 입력하기

3. 이번에는 자동 입력되는 값보다 큰 값을 입력해 보자

Do it! 자동 입력되는 값보다 큰 값을 입력한 경우

```
INSERT INTO doit_increment (col_1, col_2, col_3) VALUES (5, '4 건너뛰고 5 자동  
입력', 5);
```

```
SELECT * FROM doit_increment;
```

실행 결과

	col_1	col_2	col_3
▶	1	1 자동 입력	1
	2	2 자동 입력	2
	3	3 자동 입력	3
	5	4 건너뛰고 5 자동 입력	5

이 경우도 문제없이 쿼리가 실행되고
이와 같은 결과가 출력됨

▪ AUTO_INCREMENT로 데이터 자동 입력하기

▪ AUTO_INCREMENT로 데이터 입력하기

4. 다시 AUTO_INCREMENT이 적용된 1열을 제외하고 2, 3열에 데이터를 입력해 보자. 어떤 값이 입력될까?

Do it! 1열을 제외하고 데이터 입력한 경우

```
INSERT INTO doit_increment (col_2, col_3) VALUES ('어디까지 입력되었을까?', 0);
```

```
SELECT * FROM doit_increment;
```

5 다음 값인 6이 자동 입력된 것을 알 수 있음

실행 결과

	col_1	col_2	col_3
▶	1	1 자동 입력	1
	2	2 자동 입력	2
	3	3 자동 입력	3
	5	4 건너뛰고 5 자동 입력	5
	6	어디까지 입력되었을까?	0

▪ AUTO_INCREMENT로 데이터 자동 입력하기

▪ AUTO_INCREMENT로 자동 생성된 마지막 값 확인하기

- 앞선 예처럼 데이터를 입력하다 보면
숫자가 연속적으로 입력되는 것이 아니므로
AUTO_INCREMENT가 적용됐을 때 데이터가 어디까지 입력됐는지 궁금할 것임.

이때 다음 쿼리를 사용하면 AUTO_INCREMENT가 적용된 열의
마지막 데이터를 확인할 수 있음.

Do it! AUTO_INCREMENT가 적용된 열의 마지막 데이터 조회

```
SELECT LAST_INSERT_ID();
```

실행 결과

	LAST_INSERT_ID()
▶	6

쿼리를 실행한 결과, 6까지 입력되어 있음을 알 수 있음.
LAST_INSERT_ID()는 현재 마지막 AUTO_INCREMENT 값을 보여줌

- **AUTO_INCREMENT로 데이터 자동 입력하기**

- **AUTO_INCREMENT 시작값 변경하기**

- 이번에는 AUTO_INCREMENT을 다시 적용했을 때 다른 값으로 시작하도록 설정하는 방법을 알아보자.

이미 해당 열에 입력된 값이 있다면
그 값보다는 큰 값으로 설정해야 함.

100부터 다시 시작하도록 값을 설정함

- **AUTO_INCREMENT로 데이터 자동 입력하기**
 - **AUTO_INCREMENT 시작값 변경하기**

Do it! 자동으로 입력되는 값을 100부터 시작

```
ALTER TABLE doit_increment AUTO_INCREMENT=100;  
INSERT INTO doit_increment (col_2, col_3) VALUES ('시작값이 변경되었을까?', 0);  
  
SELECT * FROM doit_increment;
```

실행 결과			
	col_1	col_2	col_3
▶	1	1 자동 입력	1
	2	2 자동 입력	2
	3	3 자동 입력	3
	5	4 건너뛰고 5 자동 입력	5
	6	어디까지 입력되었을까?	0
	100	시작값이 변경되었을까?	0

▪ AUTO_INCREMENT로 데이터 자동 입력하기

▪ AUTO_INCREMENT 증감값 변경하기

- 자동으로 입력되는 값이 1이 아닌 5씩 증가하도록 수정해 보자
- 다음 쿼리를 입력하고 결과를 살펴보면 5씩 증가된 것을 확인할 수 있음
- 그런데 증감값을 수정하면 첫 데이터는 1이 증가하고, 첫 데이터를 입력한 이후 데이터부터 5씩 증가하는 것을 알 수 있음

Do it! 자동으로 입력되는 값이 5씩 증가

```
SET @@AUTO_INCREMENT_INCREMENT = 5;
INSERT INTO doit_increment (col_2, col_3) VALUES ('5씩 증가할까? (1)', 0);
INSERT INTO doit_increment (col_2, col_3) VALUES ('5씩 증가할까? (2)', 0);
```

```
SELECT * FROM doit_increment;
```

실행 결과

	col_1	col_2	col_3
	5	4 건너뛰고 5 자동 입력	5
	6	어디까지 입력되었을까?	0
	100	시작값이 변경되었을까?	0
	101	5씩 증가할까? (1)	0
	106	5씩 증가할까? (2)	0

▪ 조회 결과를 테이블에 입력하기

▪ 조회 결과를 다른 테이블에 입력하기

- 조회한 결과를 다른 테이블에 입력해 보자
- INSERT INTO ~ SELECT 또는 SELECT ~ INTO를 사용함
- 이때 반드시 입력하려는 테이블과 조회한 열의 데이터 유형이 동일해야 함
- 먼저 본격적인 실습을 위해서 다음과 같이
doit_insert_select_from과 doit_insert_select_to라는 2개의 테이블을 생성함
- 그다음, 초기 데이터를 doit_insert_select_from에 입력함
- 마지막으로 doit_insert_select_from의 데이터를 조회해서
doit_insert_select_to 테이블로 입력해 보자

- 조회 결과를 테이블에 입력하기
 - 조회 결과를 다른 테이블에 입력하기

Do it! INSERT INTO와 SELECT로 다른 테이블에 결과 입력

```
CREATE TABLE doit_insert_select_from (  
col_1 INT,  
col_2 VARCHAR(10)  
);
```

```
CREATE TABLE doit_insert_select_to (  
col_1 INT,  
col_2 VARCHAR(10)  
);
```

```
INSERT INTO doit_insert_select_from VALUES (1, 'Do');  
INSERT INTO doit_insert_select_from VALUES (2, 'It');  
INSERT INTO doit_insert_select_from VALUES (3, 'MySQL');
```

실행 결과

	col_1	col_2
▶	1	Do
	2	It
	3	MySQL

```
INSERT INTO doit_insert_select_to  
SELECT * FROM doit_insert_select_from;
```

```
SELECT * FROM doit_insert_select_to;
```


- 조회 결과를 테이블에 입력하기

- 새 테이블에 조회 결과 입력하기

- 조회한 결과를 아예 새로운 테이블을 생성해 저장할 수도 있음
 - CREATE TABLE 문을 사용할 때 열 이름을 입력하는 것 대신
SELECT 문을 사용하여 테이블을 생성하는 쿼리를
다음과 같이 넣어 데이터 조회 결과를 바로 새 테이블에 입력할 수 있음

- 조회 결과를 테이블에 입력하기

- 새 테이블에 조회 결과 입력하기

Do it! INSERT INTO와 SELECT로 다른 테이블에 결과 입력

```
CREATE TABLE doit_select_new AS (SELECT * FROM  
doit_insert_select_from);
```

```
SELECT * FROM doit_select_new;
```

실행 결과		
	col_1	col_2
▶	1	Do
	2	It
	3	MySQL

- 결과를 살펴보면 doit_select_new라는 새로운 테이블이 생성되어 해당 테이블에 doit_insert_select_from 테이블을 조회한 결과가 저장된 것을 확인할 수 있음

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 조작하기

- 관계형 데이터베이스는 데이터의 무결성을 유지해야 하므로 부모 테이블에 없는 데이터가 자식 테이블에 있으면 안 되는 것이 원칙임
 - 이 원칙을 유지하면 데이터가 잘못 입력되거나 삭제되는 것을 방지할 수 있음
 - 여기서는 테이블의 종속 관계가 있을 때 외래키를 가진 하위 테이블로 인해 데이터 또는 테이블 삭제에 실패하는 실습을 진행해 볼 것임
 - 이를 통해 데이터 무결성에 대해 이해해 보자

- 조회 결과를 테이블에 입력하기
 - 외래키로 연결되어 있는 테이블 조작하기

회원			주문			
회원 번호	회원 이름	회원 주소	주문 번호	회원 번호	주문 상품	배송 주소
1000	이지스	서울	100	1000	Do it! SQL	서울
1001	퍼블리싱	대전	101	1001	Do it! SQL	대전
1002	주식회사	제주도	102	1000	Do it! Python	서울

- 외래키는 테이블 간에 관계를 구성할 때 참조하는 열을 의미함
- 이 두 테이블을 살펴보면 '회원' 테이블의 '회원 번호' 열은 기본키이고 '주문' 테이블의 '회원 번호' 열은 외래키임

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블에 데이터 입력 및 삭제하기

- 먼저 실습을 위해 다음과 같이 테이블 2개를 생성하는데 하나는 부모 역할을, 다른 하나는 자식 역할을 하도록 구성해 보자.

부모 테이블과 자식 테이블 생성
<pre>CREATE TABLE doit_parent (col_1 INT PRIMARY KEY); CREATE TABLE doit_child (col_1 INT); ALTER TABLE doit_child ADD FOREIGN KEY (col_1) REFERENCES doit_parent(col_1);</pre>

REFERENCES는 참조 관계를 생성하는 명령문으로
여기서는 doit_child가 doit_parent 테이블을 참조함

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블에 데이터 입력 및 삭제하기

2. 하위 테이블(자식 테이블)에 데이터를 입력할 때
상위 테이블(부모 테이블)에 해당 데이터가 없어 데이터 입력에 실패하는 쿼리임

자식 테이블에 데이터 입력 시 부모 테이블에 해당 데이터가 없는 경우
INSERT INTO doit_child VALUES (1);

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블에 데이터 입력 및 삭제하기

2. 오류를 살펴보면 다음과 같은 메시지가 출력됨.

```
Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails
(`doitsql`.`doit_child`, CONSTRAINT `doit_child_ibfk_1` FOREIGN KEY (`col_1`)
REFERENCES `doit_parent` (`col_1`))
```

오류 내용은 doit_child 테이블이 doit_parent 테이블의 col_1에 대해서 외래키가 설정되어 있어 참조 오류가 발생한 것임.

이 오류는 부모 테이블에 데이터를 먼저 입력해야 해결할 수 있음

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블에 데이터 입력 및 삭제하기

- 부모 테이블에 데이터를 입력한 뒤,
같은 데이터를 자식 테이블에 입력하면 오류 없이 결과가 출력됨

Do it! 부모 테이블에 데이터 입력 후 자식 테이블에도 데이터 입력

```
INSERT INTO doit_parent VALUES (1);  
INSERT INTO doit_child VALUES (1);
```

```
SELECT * FROM doit_parent;  
SELECT * FROM doit_child;
```

실행 결과	
	col_1
▶	1

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블에 데이터 입력 및 삭제하기

4. 이번에는 부모 테이블의 데이터만 삭제해 보자.
이 역시도 외래키 때문에 오류가 발생하며 데이터 삭제에 실패함

부모 테이블에서만 데이터를 삭제한 경우
DELETE FROM doit_parent WHERE col_1 = 1;

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블에 데이터 입력 및 삭제하기

4. 오류를 살펴보면 다음과 같은 메시지가 출력됨.

```
Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails  
(`doitsql`.`doit_child`, CONSTRAINT `doit_child_ibfk_1` FOREIGN KEY (`col_1`)  
REFERENCES `doit_parent` (`col_1`))
```

오류 내용은 doit_child 테이블이 doit_parent 테이블의 col_1에 대해서 외래키가 설정되어 있어 참조 오류가 발생한 것임.

이 오류는 자식 테이블의 데이터를 먼저 삭제하고
부모 테이블의 데이터를 삭제해야 정상적으로 데이터를 삭제할 수 있음

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블에 데이터 입력 및 삭제하기

5. 다음과 같이 쿼리를 입력해 자식 테이블 → 부모 테이블 순으로 데이터를 삭제해 보자.

Do it! 자식 테이블의 데이터 삭제 후 부모 테이블의 데이터 삭제

```
DELETE FROM doit_child WHERE col_1 = 1;  
DELETE FROM doit_parent WHERE col_1 = 1;
```

실행 결과

#	Time	Action	Message	Duration / Fetch
626	23:41:30	DELETE FROM doit_child WHERE col_1 = 1	1 row(s) affected	0.016 sec
627	23:41:30	DELETE FROM doit_parent WHERE col_1 = 1	1 row(s) affected	0.000 sec

오류 없이 원하는 데이터가 삭제되었음을 알 수 있음

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

1. 이번에는 데이터가 아니라 테이블을 삭제해 보자.

앞에서는 데이터가 부모 테이블에 있는지 여부에 따라 입력과 삭제에 영향을 받았음.

하지만 테이블을 삭제할 경우에는
테이블에 데이터 없이 외래키 설정만 되어 있어도 오류가 발생함.

여기까지 실습을 잘 따라 했다면 외래키 설정만 되어 있고
실제 데이터는 입력된 상태가 아닐 것임.

일단 부모 테이블을 삭제하는 쿼리를 입력해 보자

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

1. 부모 테이블을 삭제할 경우

```
DROP TABLE doit_parent;
```

이와 같이 오류가 발생함. 오류를 살펴보면 다음과 같은 메시지가 출력됨.

```
Error Code: 3730. Cannot drop table 'doit_parent' referenced by a foreign key constraint 'doit_child_ibfk_1' on table 'doit_child'.
```

오류 내용은 doit_parent 테이블과 doit_child 테이블이 외래키로 참조되어 있어 doit_parent 테이블 삭제에 실패했다는 내용임

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

2. 외래키가 설정되어 있는 테이블을 삭제하려면
자식 테이블 → 부모 테이블 순서로 삭제해야 함.

그런데 자식 테이블의 데이터는 유지하면서 부모 테이블을 삭제하고 싶다면
제약 조건을 제거하여 부모 테이블 삭제를 진행하는 방법도 있음.

여기서는 그 두 가지 방법을 모두 실습해 보자

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

2. 먼저, 자식 테이블을 삭제하고 부모 테이블을 삭제하는 쿼리를 입력해 보자.

하위 테이블 삭제 후 상위 테이블 삭제
DROP TABLE doit_child; DROP TABLE doit_parent;

오류 없이 테이블이 삭제됨

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

3. 이번에는 제약 조건을 제거하여 부모 테이블을 삭제해 보자.

먼저, 제약 조건을 확인해 보자. 제약 조건을 확인하고 싶다면 SHOW CREATE TABLE을 실행하면 확인할 수 있음.

다시 참조 관계의 doit_parent, doit_child 테이블을 생성함.

그다음, SHOW CREATE TABLE을 입력하여 해당 테이블의 제약 조건을 확인한 것임

- 조회 결과를 테이블에 입력하기
- 외래키로 연결되어 있는 테이블 삭제하기

3. **Do it!** 부모 테이블 생성 후 제약 조건 확인

```
CREATE TABLE doit_parent (col_1 INT PRIMARY KEY);  
CREATE TABLE doit_child (col_1 INT);  
  
SHOW CREATE TABLE doit_child;
```

실행 결과

#	Time	Action	Message	Duration / Fetch
629	23:42:24	DROP TABLE doit_parent	Error Code: 3730. Cannot drop table 'doit_parent' referenced by a foreign key constraint '...	0.000 sec

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

3. 제약 조건을 자세히 살펴보면 다음과 같음.

```
CREATE TABLE `doit_child` (  
    `col_1` int DEFAULT NULL,  
    KEY `col_1` (`col_1`),  
    CONSTRAINT `doit_child_ibfk_1` FOREIGN KEY (`col_1`) REFERENCES  
    `doit_parent` (`col_1`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

출력 결과에서 CONSTRAINT `doit\_child\_ibfk\_1` FOREIGN KEY (`col\_1`) REFERENCES `doit\_parent` (`col\_1`) 부분이 바로 제약 조건을 나타내고 있으며, 여기서는 외래키 제약 조건이 있다는 의미임

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

4. 제약 조건을 확인했으니 제약 조건을 제거한 뒤, 부모 테이블을 삭제해 보자.
DROP CONSTRAINT 외래키를 입력하여 제약 조건을 제거한 뒤,
부모 테이블인 doit_parent를 삭제하면 오류 없이 테이블이 제거됨

제약 조건 제거 후 상위 테이블 삭제
ALTER TABLE doit_child DROP CONSTRAINT doit_child_ibfk_1; DROP TABLE doit_parent;

- 조회 결과를 테이블에 입력하기

- 외래키로 연결되어 있는 테이블 삭제하기

5. 일시적으로 외래키 설정을 OFF하고 데이터를 입력이나 수정할 수도 있음.

SET 문으로 설정하면 되는데,
외래키 설정을 OFF한 상태에서 데이터를 변경한 후
외래키를 다시 ON하면 됨.

이때, 이미 입력된 데이터에 대해서는 외래키 체크를 하지 않음.

즉 원래 외래키는 데이터의 무결성을 유지하기 위해 사용되는데,
OFF 상태에서는 무결성을 깨고 데이터를 입력해도
다시 ON 상태로 되돌아갔을 때 입력된 데이터에 대한 무결성 검사를 하지 않음

- 조회 결과를 테이블에 입력하기
- 외래키로 연결되어 있는 테이블 삭제하기

5.

외래키 비활성

```
SET FOREIGN KEY CHECKS=0
```

외래키 활성화

```
SET FOREIGN KEY CHECKS=1
```

- 앞에서 조회한 결과를 다른 테이블에 입력할 때
입력하려는 테이블과 조회한 열의 데이터 유형이 동일해야
오류가 발생하지 않는다고 했음
- MySQL은 숫자형, 문자형, 날짜형, 지리형, 공간형 등
다양한 데이터 유형을 제공하는데,
이번에는 데이터 유형인 숫자형, 문자형, 날짜형을 더 자세히 알아보자

■ 숫자형 알아보기

■ 정수형

- 정수는 기본적으로 부호를 갖는 SIGNED 형식으로 저장되어 음수와 양수를 동시에 저장할 수 있음.

부호를 갖지 않는 UNSIGNED라는 형식도 사용할 수 있는데, 이 형식으로 설정되면 0보다 큰 양의 정수를 더 많이 저장할 수 있어 최댓값은 SIGNED 형식보다 2배 커짐.

AUTO_INCREMENT가 적용되는 열에는 음수를 저장할 수 없으므로 UNSIGNED를 명시하면 작은 데이터 공간에 더 많은 값을 저장할 수 있음.

SIGNED나 UNSIGNED에 따른 성능 문제는 없지만 최솟값과 최댓값의 범위가 다르므로 사용할 때 데이터 오류가 발생하지 않도록 주의하자

■ 숫자형 알아보기

■ 정수형

- 각 정수형의 데이터 크기와 범위

데이터 유형	데이터 크기(byte)	숫자 범위(SIGNED)	숫자 범위(UNSIGNED)
TINYINT	1	-128 ~ 127	0 ~ 255
SMALLINT	2	-32,768 ~ 32,767	0 ~ 65535
MEDIUMINT	3	-8388608 ~ 8388607	0 ~ 16777215
INT	4	-2 ³¹ (약 -21억) ~ 2 ³¹ -1(약 21억)	0 ~ 2 ³² -1(약 42억)
BIGINT	8	-2 ⁶³ ~ 2 ⁶³ -1	0 ~ 2 ⁶⁴ -1

매우 큰 숫자값이나 고정 소수점 숫자를 저장해야 하는 것이 아니라면
일반적으로 INT나 BIGINT을 자주 사용함

- 숫자형 알아보기

- 실수형

- MySQL에서는 고정 소수점 형식과 부동 소수점 형식을 사용할 수 있음
 - 부동 소수점 숫자를 저장하기 위해 FLOAT와 DOUBLE을 사용할 수 있음
 - 고정 소수점 숫자를 저장하기 위해서는 DECIMAL와 NUMBER을 사용할 수 있음

■ 숫자형 알아보기

■ 실수형

- 부동 소수점(floating point)에서 '부동'이란 소수점의 위치가 고정적이지 않다는 뜻으로, 숫자 값의 길이에 따라 유효 범위의 소수점 자리가 바뀜
- 이러한 이유로 부동 소수점을 사용하면 정확한 유효 소수점의 값을 식별하기 어렵고, 부동 소수점으로 된 값끼리 비교하기가 쉽지 않음
- 또한 동등 비교를 사용할 수 없는 제약도 있음

■ 숫자형 알아보기

■ 실수형

- 실습을 위해 doit_float 테이블을 생성해 보자
- col_1열을 생성할 때 데이터 유형을 FLOAT로 지정함
- 그리고 0.7을 입력 후 조회해 보자

Do it! 실수형 데이터가 있는 테이블 생성

```
USE doitsql;  
CREATE TABLE doit_float (col_1 FLOAT);  
INSERT INTO doit_float VALUES (0.7);  
  
SELECT * FROM doit_float WHERE col_1 = 0.7;
```

실행 결과

col_1
0.7

■ 숫자형 알아보기

■ 실수형

- FLOAT 타입은 부동 소수점 숫자를 저장하기 때문에 정확한 값이 아닌 근사치를 저장함
- 0.7을 입력했을 때 0.7과 정확히 일치하는 값이 없기 때문에 이와 같이 빈 결과를 반환하게 된 것임
- FLOAT와 DOUBLE은 부동 소수점을 갖는 숫자 데이터에 사용하는 데이터 유형으로, FLOAT는 기본적으로 4바이트를 사용하여 유효 자릿수 8개를 유지하며 정밀도가 명시된 경우에는 최대 8바이트까지 저장 공간을 사용함
- DOUBLE은 8바이트 공간을 사용하여 최대 16개의 유효 자릿수를 유지함

데이터 유형	데이터 크기(byte)	숫자 범위
FLOAT(n)	4 ~ 8	-3.40E+30 ~ 1.17E-38
DOUBLE	8	-1.22E-308 ~ 1.79E+308

■ 숫자형 알아보기

■ 실수형

- 부동 소수점에서는 유효 범위 이외의 값은 가변적이므로 정확한 값을 보장할 수 없음
- 만약 금융 데이터나 소수점이 확실히 고정되어야 하는 숫자라면 FLOAT나 DOUBLE을 사용할 수 없음
- 이 경우 DECIMAL 또는 NUMERIC을 사용함
- 전체 자릿수와 소수 자릿수가 고정된 데이터 유형으로 최대 38자리 숫자를 사용할 수 있도록 함
- 예를 들어 DECIMAL(20, 5)라고 하면 정수 15자리, 소수 5자리를 표현함

■ 숫자형 알아보기

■ 실수형

- 하지만 DECIMAL(20)이라고 표현하면 정수 20자리까지만 저장할 수 있음
- DECIMAL 형식의 숫자 데이터 하나를 저장하는 데 1/2바이트가 필요함
- 그래서 자릿수를 계산할 때 (전체 자릿수)/2의 결과값을 올림 처리한 만큼의 바이트 수가 필요함

데이터 유형	데이터 크기(byte)	숫자 범위
DECIMAL(p,s)	5 ~ 17	$-10^{38}+1 \sim 10^{38}+1$
NUMERIC(p,s)	5 ~ 17	$-10^{38}+1 \sim 10^{38}+1$

■ 숫자형 알아보기

■ 숫자형 변환

- 숫자형 데이터를 사용할 때는 형 변환에 주의해야 함
- 형 변환 type casting이란 암시적, 명시적으로 자료형을 변경하는 것임
- 암시적 형 변환이란 자료형을 직접 변경하지 않아도 실행 환경에서 자동으로 자료형을 변경하는 것을 말하고,

명시적 형 변환이란 CAST, CONVERT 등의 함수를 사용하여 사용자가 자료형을 직접 변경하는 것을 말함

숫자형 알아보기

숫자형 변환

- 다음과 같이 $10/3$ 을 계산하면 10은 정수, 3은 정수이므로 결과값은 정수인 3이 출력되어야 하지만 실수 3.3333이 출력됨
- 이 경우는 계산 결과가 10과 3과 같은 정수가 아닌 3.3333인 실수로, 즉 우선순위 형 변환 정책에 따라 암시적 형 변환이 발생한 것임

Do it! 암시적 형 변환으로 계산 결과가 출력된 예

```
SELECT 10/3;
```

실행 결과

	10/3
▶	3.3333

- 숫자형 알아보기

- 숫자형 변환

- 이와 같이 일치하지 않는 데이터 유형은 MySQL에서 호환 가능한 형식으로 암시적으로 변환되는데, 우선순위가 정의된 프로세스에 따라 수행됨
 - 우선순위가 낮은 데이터 유형은 우선순위가 높은 데이터 유형으로 변환됨

■ 숫자형 알아보기

■ 숫자형 변환

- 마이크로소프트 공식 문서에서 제공하는 데이터 우선순위
- MySQL은 공식 문서를 확인할 수 없는데, 관계형 데이터베이스는 국제 표준에 따라 비슷하게 동작하는 부분이 많아서 이 내용을 참고하도록 하자

순위	데이터 유형	순위	데이터 유형	순위	데이터 유형
1	user-defined data types	11	real	21	text
2	sql_variant	12	decimal	22	image
3	xml	13	money	23	timestamp
4	datetimeoffset	14	smallmoney	24	uniqueidentifier
5	datetime2	15	bigint	25	nvarchar (including nvarchar(max))
6	datetime	16	int	26	nchar
7	smalldatetime	17	smallint	27	varchar (including varchar(max))
8	date	18	tinyint	28	char
9	time	19	bit	29	varbinary (including varbinary(max))
10	float	20	ntext	30	binary

■ 숫자형 알아보기

■ 숫자형 변환

- 숫자형 데이터는 합, 평균, 나누기 등 다양한 집계 함수에서 자주 사용함
- 실무에서는 여러 보고서에 숫자형 데이터를 많이 사용하므로 데이터 처리 시 발생할 수 있는 오류를 예방하려면 자료형과 형 변환을 이해하는 것은 무척 중요함

■ 문자형 알아보기

- 문자형은 다양한 문자를 저장할 수 있는 데이터 유형
- 문자형은 크게 고정 길이와 가변 길이로 구분할 수 있음
- 고정 길이는 실제 값을 입력하지 않아도 지정한 만큼 저장 공간을 사용하며, 가변 길이는 실제 입력한 값의 크기만큼만 저장 공간을 사용함

■ 문자형 알아보기

- 다음 예를 살펴보자



- 열에 저장되는 데이터 유형이 잘못 정해지거나 저장 길이가 너무 짧은 경우 불가피하게 열의 길이를 변경하는 작업이 필요할 수도 있음
- 이러한 스키마 변경 작업은 현재 운영중인 시스템을 중단해야 하는 등 부담스런 일이 될 수도 있으므로 실제로 저장되는 데이터의 성격을 파악하고 최적의 데이터 유형과 길이를 선정하는 것이 매우 중요함

■ 문자형 알아보기

- 문자열 데이터의 길이가 항상 고정이라면
(예를 들어 주민등록번호나 전화번호 같이
변화가 없는 자릿수를 가진 데이터인 경우) CHAR를 사용하고,

글자 길이를 예측할 수 없지만
최대 글자 수를 예측할 수 있을 때에는 VARCHAR를 사용함
- 하지만 입력되는 문자열이 2~3바이트 정도의 범위 내에서
예측하기 어려운 경우에는 넉넉한 CHAR로 설정하는 것이 좋음

■ 문자형 알아보기

- 고정 길이 문자열은 미리 길이가 정해져 있기 때문에 실제 입력되는 값에 따라 저장 공간의 크기가 변하지 않음
- 하지만 가변 길이 문자열은 최대 저장할 수 있는 값의 길이는 제한하고 제한된 길이 이하 크기의 값이 저장되면 그만큼 저장 공간의 크기가 줄어듦
- 가변 길이에 저장된 값의 유효 크기가 얼마인지를 관리하기 위해서는 별도의 저장 공간이 필요하기 때문에 고정 길이 문자열에 비해 1~2바이트의 저장 공간이 추가로 더 필요함

■ 문자형 알아보기

데이터 유형		데이터 크기(byte)	설명
CHAR(n)		1 ~ 255	고정 길이 문자열로, 0 ~ 255 크기의 문자열까지 저장 가능 기본값
VARCHAR(n)		1 ~ 65535	가변 길이 문자열로, n만큼의 크기를 지정. 0 ~ 16383 크기의 문자열까지 저장 가능(한 문자당 최대 4바이트 사용)
BINARY(n)		1 ~ 255	고정 길이 문자열로, 0 ~ 255 크기의 문자열까지 저장 가능
VARBINARY(n)		1 ~ 65535	가변 길이 문자열로, 0 ~ 16383 크기의 문자열까지 저장 가능
TEXT 형식	TINYTEXT	1 ~ 255	최대 길이가 255자인 문자열을 저장
	TEXT(n)	1 ~ 65535	최대 길이가 65,535바이트인 문자열 데이터 저장
	MEDIUMTEXT	1 ~ 16777215	최대 길이가 16,777,215바이트인 문자열 데이터 저장
	LONGTEXT	1 ~ 4294967295	최대 길이가 4,294,967,295바이트인 문자열 데이터 저장
BLOB 형식	TINYBLOB	1 ~ 255	최대 길이가 255바이트인 문자열 데이터 저장
	BLOB(n)	0 ~ 65535	최대 길이가 65,535바이트인 데이터 저장
	MEDIUMBLOB	1 ~ 16777215	최대 길이가 16,777,215바이트인 문자열 데이터 저장
	LOBLOB	1 ~ 4294967295	최대 길이가 4,294,967,295바이트인 문자열 데이터 저장
ENUM		1 ~ 2	가능한 값 목록에서 선택한 하나의 값만 가질 수 있는 문자열 개체로, ENUM 목록에 최대 65535개의 값을 나열할 수 있다.
SET		1, 2, 3, 4, 8	가능한 값 목록에서 선택한 0개 이상의 값을 가질 수 있는 문자열 개체로, 최대 64개의 값을 나열할 수 있다.

▪ 문자형 알아보기

- MySQL의 경우 문자열 데이터 유형을 사용할 때, CHAR 또는 VARCHAR 뒤에 인자(괄호 안의 숫자)로 사용하는 숫자값의 의미는 다른 DBMS와 달라서 정확히 알고 사용해야 함
- MySQL에서는 CHAR 또는 VARCHAR 뒤에 나오는 숫자는 바이트가 아닌 문자열의 문자 수임
- 즉, CHAR(5) 또는 VARCHAR(5)이면 5byte를 저장할 수 있는 것이 아니라 문자 5개를 저장할 수 있다는 뜻임
- 그래서 CHAR(5)을 사용하더라도 실제 저장 공간은 입력되는 데이터에 따라 크기가 다른 공간을 사용함

■ 문자형 알아보기

- 다음 예제를 작성하여 이 내용을 이해해 보자

Do it! 문자열 데이터의 길이와 크기 확인

```
USE doitsql;
CREATE TABLE doit_char_varchar (
col_1 CHAR(5),
col_2 VARCHAR(5)
);

INSERT INTO doit_char_varchar VALUES ('12345', '12345');
INSERT INTO doit_char_varchar VALUES ('ABCDE', 'ABCDE');
INSERT INTO doit_char_varchar VALUES ('가나다라마', '가나다라마');
INSERT INTO doit_char_varchar VALUES ('hello', '안녕하세요');
INSERT INTO doit_char_varchar VALUES ('안녕안녕안', '안녕안녕안');

SELECT
col_1, CHAR_LENGTH(col_1) as char_length, LENGTH(col_1) AS
char_byte,
col_2, CHAR_LENGTH(col_2) as char_length, LENGTH(col_2) AS
char_byte
FROM doit_char_varchar;
```

실행 결과

	col_1	char_length	char_byte	col_2	char_length	char_byte
▶	12345	5	5	12345	5	5
	ABCDE	5	5	ABCDE	5	5
	가나다라마	5	15	가나다라마	5	15
	hello	5	5	안녕하세요	5	15
	안녕안녕안	5	15	안녕안녕안	5	15

▪ 문자형 알아보기

- 결과를 살펴보면 숫자와 영어는 한 글자당 1바이트, 그리고 한글과 한문은 한 글자당 3바이트를 사용한 것을 확인할 수 있음
- 그런데 다른 DBMS를 살펴보면 숫자, 영어 이외의 문자는 유니코드라고 하여 2바이트를 사용한다고 함
- MySQL에서 3바이트를 사용하는 이유는 MySQL에서는 더 다양한 문자 집합을 사용하기 때문임

▪ 문자형 알아보기

- MySQL에서는 유니코드 UTF-8은 3바이트를, UTF-16은 2바이트를 사용함
- 최근에는 다양한 이모티콘 등을 표현하기 위해 UTF-8MB4라는 형식을 기본으로 사용하기도 함
- 따라서 이와 같이 문자 집합별로 사용하는 데이터 크기가 다양하기 때문에 실제 문자열의 개수와 사용되는 물리적 바이트 수가 달라질 수도 있음

▪ 문자형 알아보기

▪ 문자 집합별로 사용하는 저장 공간

문자 집합별 사용하는 저장 공간의 크기

- 아스키 문자: 1바이트의 저장 공간을 사용
- 추가 알파벳 문자: 2바이트의 저장 공간을 사용
- BMP 문자: 3바이트의 저장 공간을 사용
- SMP, SIP 문자 등: 4바이트의 저장 공간을 사용

■ 문자형 알아보기

■ 문자 집합

- 문자형을 배울 때 반드시 알아 두어야 할 개념
- MySQL에서는 데이터베이스뿐만 아니라 테이블, 열까지 모두 서로 다른 문자 집합(character set, 캐릭터셋)을 사용할 수 있지만 문자열을 저장하는 CHAR, VARCHAR, TEXT이 적용된 열에만 사용할 수 있음
- 일반적으로 한글은 EUC-KR 또는 UTF8MB4 문자 집합을 사용하고, 일본어는 CP932 또는 UTF8MB4 등을 사용함

■ 문자형 알아보기

■ 문자 집합

- MySQL 서버에서 사용할 수 있는 문자 집합은 다음 쿼리로 확인할 수 있으며 여러 가지 문자 집합을 사용한 것을 확인할 수 있음
- 보통 UTF8MB4를 사용하면 문제없이 전 세계 문자 및 이모티콘을 저장할 수 있음

Do it! MySQL의 문자 집합 확인

```
SHOW CHARACTER SET;
```

실행 결과

Charset	Description	Default collation	Maxlen
macce	Mac Central European	macce_general_ci	1
macroman	Mac West European	macroman_general_ci	1
sjis	Shift-JIS Japanese	sjis_japanese_ci	2
swe7	7bit Swedish	swe7_swedish_ci	1
tis620	TIS620 Thai	tis620_thai_ci	1
ucs2	UCS-2 Unicode	ucs2_general_ci	2
ujis	EUC-JP Japanese	ujis_japanese_ci	3
utf16	UTF-16 Unicode	utf16_general_ci	4
utf16le	UTF-16LE Unicode	utf16le_general_ci	4
utf32	UTF-32 Unicode	utf32_general_ci	4
utf8mb3	UTF-8 Unicode	utf8mb3_general_ci	3
utf8mb4	UTF-8 Unicode	utf8mb4_0900_ai_ci	4

■ 문자형 알아보기

■ 콜레이션

- 콜레이션(collation)은 문자열 데이터가 담긴 열의 비교나 정렬 순서를 위한 규칙
- 즉, 콜레이션에 따라 우선순위를 한글로 할지, 영어로 할지, 대소 문자를 구분할지 등이 결정됨
- 앞서 ORDER BY 문을 사용하면 데이터가 정렬되어 반환된다고 설명함
- 이때 ORDER BY에서는 정렬할 때 단순히 오름차순(ASC) 또는 내림차순(DISC) 정의만 할 수 있었음
- 그런데 오름차순, 내림차순 등 정렬 우선순위라고 하는 콜레이션이 이미 적용된 것임
- 한 데이터베이스 또는 테이블 안에서 혹은 열 안에서 콜레이션이 다를 경우 정렬 작업을 한 결과가 달라질 수 있으므로 매우 주의해야 함

■ 문자형 알아보기

■ 콜레이션

- 다음 쿼리를 통해 콜레이션에 따라 정렬 순서가 어떻게 달라지는지 확인해 보자
- 먼저 다음 내용을 입력해 테이블을 생성해 보자

Do it! 콜레이션에 따른 정렬 순서 비교를 위한 테이블 생성

```
CREATE TABLE doit_collation (  
  
col_latin1_general_ci    VARCHAR(10)    COLLATE  
latin1_general_ci,  
col_latin1_general_cs    VARCHAR(10)    COLLATE  
latin1_general_cs,  
col_latin1_bin VARCHAR(10) COLLATE latin1_bin,  
col_latin7_general_ci    VARCHAR(10)    COLLATE  
latin7_general_ci  
);  
INSERT INTO doit_collation VALUES ('a', 'a', 'a', 'a');  
INSERT INTO doit_collation VALUES ('b', 'b', 'b', 'b');  
INSERT INTO doit_collation VALUES ('A', 'A', 'A', 'A');  
INSERT INTO doit_collation VALUES ('B', 'B', 'B', 'B');  
INSERT INTO doit_collation VALUES ('*', '*', '*', '*');  
INSERT INTO doit_collation VALUES ('_', '_', '_', '_');  
INSERT INTO doit_collation VALUES ('!', '!', '!', '!');  
INSERT INTO doit_collation VALUES ('1', '1', '1', '1');
```

■ 문자형 알아보기

■ 콜레이션

- 다음 쿼리를 한 줄씩 실행하며 정렬한 결과를 비교해 보자.
그러면 콜레이션에 따라 정렬 순서가 다른 것을 확인할 수 있음

Do it! 콜레이션에 따른 정렬 순서 확인

```
SELECT col_latin1_general_ci FROM doit_collation ORDER BY  
col_latin1_general_ci;
```

```
SELECT col_latin1_general_cs FROM doit_collation ORDER BY  
col_latin1_general_cs;
```

```
SELECT col_latin1_bin FROM doit_collation ORDER BY col_latin1_bin;
```

```
SELECT col_latin7_general_ci FROM doit_collation ORDER BY  
col_latin7_general_ci;
```

■ 문자형 알아보기

■ 콜레이션

- 한 줄씩 실행한 쿼리 결과와 함께 다음 내용을 살펴보자

- latin1_general_ci 콜레이션으로 정렬된 알파벳 데이터들을 살펴보면 대소 문자를 구분하지 않고 정렬된 것을 확인할 수 있다.
- latin1_general_cs 콜레이션으로 정렬된 데이터 중 대문자 A와 소문자 a는 모두 대문자 B보다 먼저 정렬된다. 그리고 같은 알파벳에서는 대문자가 소문자보다 먼저 정렬된다.
- latin1_bin 콜레이션으로 정렬된 데이터를 살펴보면 대문자가 먼저 정렬되고 소문자가 정렬된다. 그리고 특수 문자가 알파벳 사이에 정렬된 것을 확인할 수 있다.
- latin7_general_ci 콜레이션으로 정렬된 데이터 중 특수 문자는 항상 알파벳보다 먼저 정렬된다.

▪ 문자형 알아보기

▪ 콜레이션

- 이처럼 콜레이션에 따라 정렬 우선순위가 달라져서 데이터를 비교할 때 크다, 작다를 판단하는 데 예상치 못한 결과를 반환할 수 있음
- 이는 정렬 순서에도 영향을 미침
- 데이터베이스 또는 테이블을 생성할 때 특별한 경우를 제외하고는 MySQL 기본 설정값을 사용하기 때문에 동일한 데이터베이스 내에서 정렬 순서로 인한 큰 문제는 없음

■ 날짜형 및 시간형 알아보기

- 날짜 또는 시간만 저장하거나 날짜와 시간을 함께 저장하는 데이터 유형도 있음
- 어떤 형태로 데이터를 저장하느냐에 따라 그 크기와 형식이 조금씩 다름

데이터 유형	데이터 크기(byte)	설명
TIME	3	HH:MM:SS(시:분:초) 형태의 데이터에 사용하고, 범위는 -838:59:59 ~ 838:59:59이다.
DATE	3	날짜만 있는 데이터에 사용하고, YYYY-MM-DD(연-월-일) 형태이다. 범위는 1000-01-01 ~ 9999-12-31이다.
DATETIME	8	날짜와 시간을 모두 포함하는 데이터에 사용하고, YYYY-MM-DD hh:mm:ss [.fraction] 형식으로, 범위는 1000-01-01 00:00:00.000000 ~ 9999-12-31 23:59:59.000000이다.
TIMESTAMP	4	날짜와 시간 부분을 모두 포함하는 데이터에 사용하고, 범위는 1970-01-01 00:00:01 UTC ~ 2038-01-19 03:14:07 UTC이다.

▪ 날짜형 및 시간형 알아보기

- DATETIME과 TIMESTAMP는 언뜻 보기엔 유사한 거 같지만 큰 차이가 있음
- DATETIME은 문자형이고, TIMESTAMP는 숫자형임
- 그래서 DATETIME은 8byte를 사용하고, TIMESTAMP는 4byte를 사용함
- 또한 DATETIME은 직접 날짜와 시간을 입력한 데이터의 형식이지만, TIMESTAMP는 데이터를 저장할 때 자동으로 입력된 현재의 UTC 날짜 데이터의 형식임

■ 날짜형 및 시간형 알아보기

- 다음 쿼리는 날짜 형식의 테이블을 생성한 다음,
현재 시간을 각 열에 동일하게 입력하여 데이터가 어떻게 저장되는지 확인함
- 정확도에 따라 초 단위에서 소수점 자릿수가 다름

Do it! 데이터 유형에 따른 현재 시간 조회

```
CREATE TABLE date_table(  
  justdate DATE,  
  justtime TIME,  
  justdatetime DATETIME,  
  justtimestamp TIMESTAMP);
```

```
INSERT INTO date_table VALUES (now(), now(), now(),  
now());
```

```
SELECT * FROM date_table;
```

실행 결과

	JustDate	JustTime	JustDateTime	JustTimeStamp
▶	2023-08-15	19:45:17	2023-08-15 19:45:17	2023-08-15 19:45:17

- 날짜형 및 시간형 알아보기

- 문자형과 날짜형 변환

- 문자 및 날짜 형식도 형 변환을 할 수 있음
 - 다음 표는 문자 형 변환을 할 수 있는 데이터 유형을 나타낸 것으로, 행의 데이터 유형이 열의 데이터 유형으로 각각 변환할 수 있는지 여부를 나타냄

- 날짜형 및 시간형 알아보기

- 문자형과 날짜형 변환

from to	DATETIME	FLOAT	DECIMAL	INT	BIT	NVARCHAR	VARCHAR
DATETIME		×	×	×	×	○	○
FLOAT	○		○	○	○	○	○
DECIMAL	○	○		○	○	○	○
INT	○	○	○		○	○	○
BIT	○	○	○	○		○	○
NVARCHAR	○	○	○	○	○		○
VARCHAR	○	○	○	○	○	○	

감사합니다

